QuickBite

An Online Food Ordering and Delivery Platform

This project report is submitted to

Silicon University, Odisha

In partial fulfilment of the requirement for the award of the degree of

Masters in Computer Application

Submitted by

Bijoy Laxmi Behera (24MMCD79)

Rojalin Mohanty(24MMCD86)

Aradhana Mohanty(24MMCD89)

4th Semester MCA (Batch: 2024-26)

Group No.: MCA07

Under the Esteemed Supervision of

Bhagwat Prasad Choudhury

Department of Computer Applications

Silicon University, Odisha

May,2026

CERTIFICATE

This is to certify that the project entitled "QuickBite – An Online Food Ordering and Delivery Platform" submitted by **Bijoy Laxmi Behera(24MMCD79), Rojalin Mohanty(24MMCD86), Aradhana Mohanty (24MMCD89)** in partial fulfilment of the requirements for the award of the Degree of Master of Computer Applications from Silicon University, Odisha, is a record of Bonafide work carried out by the student under my supervision and guidance. The project has not been submitted earlier to this or any other institution for the award of any degree or diploma.

Date:

Place:

Prof. (Dr) Bhagwat Prasad Choudhury

Sr. Assistant Professor

Department of CSE

Department of Computer Applications

Silicon University, Odisha

DECLARATION

We student of Master of Computer Applications at Silicon University, Odisha, hereby declare that the project entitled "QuickBite – An Online Food Ordering and Delivery Platform" has been prepared by us and is a result of our own work. This project has not been submitted to any other institution for the award of any degree or diploma. All information and data used in the project are true to the best of my knowledge.

Date:

Place:

Bijoy Laxmi Behera (24MMCD79)

Rojalin Mohanty (24MMCD86)

Aradhana Mohanty(24MMCD89)

Department of Computer Applications

Silicon University, Odisha

ACKNOWLEDGEMENT

We would like to express my sincere gratitude to all those who have supported and guided me throughout the development of this project. First and foremost, I am deeply thankful to my Project Guide Prof. (Dr) Bhagwat Prasad Chaudhary, for providing continuous support, valuable suggestions, and expert guidance throughout this project.

I sincerely thank the Head of the Department of Master in Computer Application, Silicon University, for providing the necessary infrastructure and academic environment for completing this project.

I would also like to thank all the faculty members of the Department of MCA for their encouragement and support during my academic journey.

I am grateful to my family and friends for their constant motivation and support throughout this endeavour.

Bijoy Laxmi Behera (24MMCD79)

Rojalin Mohanty(24MMCD86)

Aradhana Mohanty(24MMCD89)

Department of Computer Applications

Silicon University, Odisha

ABSTRACT

QuickBite is an online food ordering and delivery platform developed to simplify and automate the process of ordering food from restaurants and cloud kitchens. The system enables customers to browse restaurants and menus digitally, place orders, make secure online payments through multiple payment modes including UPI, Credit/Debit Cards, and Cash on Delivery, and track deliveries in real time. Restaurants and cloud kitchens can efficiently manage menus, monitor order queues, track inventory, and analyse sales performance, while administrators can monitor platform activities and generate analytical reports.

The platform is built using a modern full-stack technology stack consisting of React.js for the frontend, Node.js and Express.js for the backend API layer, and MongoDB for persistent data storage. The system implements JWT-based secure authentication, role-based access control for customers, restaurants, vendors, and administrators, and integrates third-party payment gateways for secure transaction processing.

QuickBite addresses the limitations of traditional food ordering methods such as long waiting times, manual order handling, lack of real-time tracking, and inefficient communication between restaurants and customers. The platform supports cloud kitchen operations which operate as delivery-only food businesses, enabling them to manage multiple brands from a single infrastructure. The project demonstrates the practical application of modern web and mobile development technologies, database management, API integration, and secure authentication in building a scalable, user-friendly digital food service ecosystem.

Keywords: *Food Ordering Platform, Cloud Kitchen, React.js, Node.js, MongoDB, REST API, JWT Authentication, Real-Time Tracking, Online Payment, Delivery Management*

Department of Computer Applications
Silicon University, Odisha

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
HTML	Hypertext Markup Language
HTTPS	Hypertext Transfer Protocol Secure
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
MCA	Master of Computer Applications
MERN	MongoDB, Express.js, React.js, Node.js
MongoDB	MongoDB Document Database
MVC	Model View Controller
OTP	One-Time Password
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
UPI	Unified Payments Interface
URL	Uniform Resource Locator
UX	User Experience
XSS	Cross-Site Scripting

LIST OF FIGURES

Page No.

LIST OF TABLES

Page No.

Chapter 2.

Chapter 3.

Chapter 4.

Chapter 5.

Chapter 6.

Chapter 7.

CONTENTS

CHAPTER 1

INTRODUCTION

QuickBite is a full-stack food delivery and meal subscription platform developed to provide a fast, convenient, and user-friendly online food ordering experience. The system includes four interfaces: Admin, Customer, Vendor, and Delivery Partner. Customers can browse restaurants, order food, manage subscriptions, and track deliveries, while vendors manage menus and orders. Delivery partners handle delivery status updates, and the admin monitors the overall platform activities. The project is developed using React.js, Node.js, Express.js, and MongoDB with a responsive modern UI inspired by popular food delivery applications. The platform aims to simplify food ordering and management through an efficient and scalable digital solution.

1.1 BACKGROUND

The rapid advancement of digital technology and the increasing penetration of smartphones and internet connectivity have transformed the way people access services in their daily lives. The food service industry, one of the most dynamic and competitive sectors of the global economy, has witnessed a significant shift towards digital platforms that enable customers to order food online without visiting a restaurant physically.

Online food delivery platforms such as Swiggy, Zomato, and Uber Eats have demonstrated the enormous potential of technology-driven food ordering systems. These platforms have revolutionized the food industry by creating a seamless ecosystem connecting customers, restaurants, and delivery partners through a single digital interface.

QuickBite is developed with the vision of providing a modern, user-friendly, and scalable online food ordering and delivery platform that addresses the needs of customers seeking convenience, restaurants seeking digital order management tools, and cloud kitchens seeking technology-driven operational support. The platform integrates features for customer order placement, restaurant menu management, real-time order tracking, subscription meal plans, and administrative monitoring.

1.2 PROBLEM STATEMENT

Traditional food ordering methods present several operational and experiential challenges for both customers and restaurant operators. Customers often face long waiting times during telephonic orders, difficulty in browsing menus, limited payment options, and lack of order visibility once an order has been placed. Restaurant operators struggle with manual order handling, inefficient communication with kitchen staff, difficulty in managing peak-hour demand, and limited digital reach to customers.

Furthermore, the emergence of cloud kitchens as a delivery-only food business model introduces additional operational complexities such as multi-brand menu management, kitchen workflow coordination, and inventory tracking without a traditional dine-in infrastructure. Existing solutions in the market are either too expensive for small restaurant operators or too simplistic to support the full lifecycle of a food ordering and delivery process.

QuickBite addresses these challenges by providing an integrated, automated, and user-friendly digital platform that manages the complete food ordering and delivery workflow from menu discovery to doorstep delivery.

1.3 OBJECTIVES

The major objectives of the QuickBite project are as follows:

1. To design and develop a user-friendly online food ordering platform accessible via web and mobile browsers.
2. To enable customers to browse restaurants, view menus, and place orders with ease.
3. To provide secure online payment integration supporting multiple payment modes.
4. To offer real-time order tracking with estimated delivery time updates.
5. To support restaurant and cloud kitchen operators in managing menus, orders, and inventory efficiently.

6. To implement role-based access control for customers, vendors, and administrators.
7. To create a scalable and maintainable platform architecture supporting future feature expansion.
8. To integrate a subscription meal plan system for recurring order management.

1.4 SCOPE OF THE PROJECT

The scope of QuickBite covers the development of a full-stack web application that serves four primary user roles: Customer, Vendor (Restaurant /Cloud Kitchen Operator), Delivery Partner, and Administrator. The platform scope includes the following functional domains:

1.4.1 Customer Scope

- User registration, login, and profile management
- Restaurant discovery, search, and category-based filtering
- Menu browsing, cart management, and order placement
- Secure online payment processing
- Real-time order tracking and delivery status updates
- Order history, ratings, reviews, and subscription management

1.4.2 Vendor Scope

- Restaurant registration and profile management
- Menu and inventory management
- Order queue management and status updates
- Sales analytics and performance tracking
- Subscription order management

1.4.3 Delivery Partner Scope

- Receiving assigned delivery requests with pickup and drop address details
- Viewing complete order and customer information for each delivery
- Updating delivery status at every stage — Assigned, Picked Up, Out for Delivery, and Delivered

- Submitting proof of delivery upon successful order completion
- Viewing personal delivery history and total deliveries completed
- Tracking own performance metrics including delivery rating and total earnings
- Receiving notifications for new delivery assignments and order updates
- Navigation support for optimized delivery routing

1.4.4 Admin Scope

- User and restaurant management
- Platform analytics and reporting
- Complaint handling and resolution

1.5 ORGANIZATION OF THE REPORT

Chapter 1 titled, *"Introduction"*, presents the general overview of the QuickBite food ordering and delivery platform. It also contains the problem statement, objectives, scope of the project, and the motivation behind the work.

Chapter 2 titled, *"Literature Review"*, presents the renowned works and existing systems earlier developed in the area of online food ordering and delivery. This chapter furthermore contains the comparative analysis and the research gap identified.

Chapter 3 named as, *"System Analysis"*, covers the feasibility study including technical, operational, and economic feasibility along with the functional and non-functional requirements of the system.

Chapter 4 titled, *"System Design"*, describes the complete system architecture, database design, entity-relationship diagram, data flow diagrams, and module-level design of the QuickBite platform.

Chapter 5 titled, *"Implementation"*, details the practical development of the system using the MERN stack. It covers the frontend, backend, database, and security implementation of the platform.

Chapter 6 titled, *"Testing"*, presents the testing strategy followed including unit testing, integration testing, system testing, and user acceptance testing along with the test case results.

Chapter 7 titled, *"Results and Discussion"*, discusses the system output screens, performance analysis, and the outcomes of the testing phase of the QuickBite platform.

Chapter 8 titled, *"Conclusion and Future Scope"*, is the summary of the complete work carried out, the key contributions of the project, and the scope for further enhancements and future development.

SUMMARY

In this chapter we discussed about:

- The concept and overview of the **QuickBite** online food ordering and delivery platform and its significance in the modern digital food service industry.
- The **problem statement** highlighting the limitations of traditional food ordering methods such as long waiting times, manual order handling, lack of real-time tracking, and inefficient communication between restaurants and customers.
- The **objectives** of the QuickBite platform including providing a user-friendly ordering interface, secure payment integration, real-time order tracking, restaurant and cloud kitchen management, and a scalable MERN stack architecture.
- The **scope of the project** covering the four primary user roles — Customer, Vendor (Restaurant /Cloud Kitchen Operator), Delivery Partner, and Administrator — and their respective functional domains within the platform.
- The **system overview** describing QuickBite as a full-stack web application built using React.js, Node.js, Express.js, and MongoDB with JWT-based authentication and role-based access control.
- The **cloud kitchen support** feature that enables delivery-only food businesses to manage multiple virtual brands from a single centralized infrastructure.
- The **organization of the report** outlining the structure and content of each subsequent chapter from Literature Review to Conclusion.

CHAPTER 2

LITERATURE REVIEW

2.1 EXISTING SYSTEMS AND RELATED WORK

Several researchers and industry platforms have explored the domain of online food ordering and delivery systems. This chapter reviews the most relevant existing systems and academic works to identify the research gap that QuickBite addresses.

a) Chopade and Dhore (2018) proposed a web-based food ordering system for college campuses that reduced manual order processing time by 60%. Their system demonstrated the viability of browser-based food ordering but lacked real-time delivery tracking and payment integration.

b) Singh and Kaur (2019) developed a mobile food ordering application using Android native development. The system supported restaurant browsing and order placement but was limited to a single platform and did not support cloud kitchen management.

c) Kumar et al. (2020) presented a RESTful API-based food delivery system that integrated GPS-based delivery tracking. Their work demonstrated the importance of real-time communication in delivery systems but did not address subscription-based meal planning.

d) Sharma and Patel (2021) studied the adoption of cloud kitchen business models in metropolitan areas and identified key technology requirements including multi-brand management, centralized order queuing, and kitchen performance analytics.

e) Verma et al. (2022) conducted a comparative analysis of popular food delivery platforms including Swiggy, Zomato, and Dunzo from an architectural and user experience perspective. Their study highlighted the critical role of responsive design, fast page loading, and intuitive navigation in user retention.

2.2 COMPARATIVE ANALYSIS OF EXISTING APPLICATIONS

Table 2.1 presents a comparative analysis of the major food delivery applications available in the Indian market in terms of key features.

Table 2.1: Comparison of Existing Food Delivery Applications

Feature	Zomato	Swiggy	Uber Eats	Dunzo	QuickBite
Real-time Tracking	✓	✓	✓	✓	✓
Cloud Kitchen Support	✓	✓	✗	✗	✓
Subscription Plans	✓	✓	✗	✗	✓
Multi-Payment Modes	✓	✓	✓	✓	✓
Vendor Dashboard	✓	✓	✓	✗	✓
Offline/Local Focus	✗	✗	✗	✓	✓

2.3 RESEARCH GAP

Based on the review of existing systems and literature, the following research gaps have been identified that QuickBite addresses:

- Most academic implementations lack integration of subscription-based meal plans with cloud kitchen support in a single platform.
- Existing open-source food ordering systems do not provide a complete vendor analytics dashboard with inventory management.
- The combination of React.js-based responsive frontend with MERN stack backend specifically designed for cloud kitchen operations has not been explored extensively in academic implementations.

- Role-based access control with separate dashboards for customers, vendors, and administrators in a unified platform is underrepresented in educational project implementations.

SUMMARY

In this chapter we discussed about:

- The **existing systems** and related academic works in the domain of online food ordering and delivery platforms, including studies by Chopade and Dhore (2018), Singh and Kaur (2019), Kumar et al. (2020), Sharma and Patel (2021), and Verma et al. (2022).

- The **comparative analysis** of major food delivery applications available in the Indian market including Zomato, Swiggy, Uber Eats, and Dunzo, evaluated on the basis of key features such as real-time tracking, cloud kitchen support, subscription plans, multi-payment modes, and vendor dashboard availability.

- The **limitations of existing systems** such as lack of integrated subscription meal plans, absence of cloud kitchen management support, limited vendor analytics dashboards, and insufficient role-based access control in academic implementations.

- The **research gap** identified from the literature review that QuickBite addresses — specifically the absence of a unified platform combining cloud kitchen management, subscription-based meal plans, and a comprehensive vendor analytics dashboard using the MERN stack.

- The **justification** for developing QuickBite as a novel academic implementation that fills the identified gaps in existing food ordering and delivery systems.

CHAPTER 3

SYSTEM ANALYSIS

3.1 FEASIBILITY STUDY

3.1.1 Technical Feasibility

The technical feasibility of QuickBite was assessed by evaluating the availability and maturity of the proposed technology stack. React.js is a production-grade frontend library maintained by Meta with a large ecosystem of supporting libraries. Node.js and Express.js provide a well-established backend framework for building RESTful APIs. MongoDB is a widely used NoSQL database with strong support for document-based data models that are well-suited for dynamic menu and order structures. All selected technologies are open-source, well-documented, and supported by large developer communities. The technical feasibility is confirmed.

3.1.2 Operational Feasibility

From an operational perspective, the web-based nature of QuickBite ensures accessibility across all modern browsers without requiring device-specific installations. The responsive design accommodates both desktop and mobile users. Restaurant operators and administrators require minimal technical knowledge to use the system as the interfaces are designed to be intuitive and user-friendly. Operational feasibility is confirmed.

3.1.3 Economic Feasibility

QuickBite is developed entirely using open-source technologies and tools, eliminating licensing costs. Deployment can be hosted on cloud platforms such as AWS, Firebase, Render, or Heroku with free tiers available for development and demonstration. For a production environment, the estimated monthly cloud infrastructure cost is within a reasonable budget for small to medium-scale deployments. Economic feasibility is confirmed.

3.2 FUNCTIONAL REQUIREMENTS

3.2.1 Customer Module

- FR1.1 – The system shall allow customers to register with email or mobile number and create a profile.
- FR1.2 – The system shall allow customers to browse restaurants by category, rating, and location.
- FR1.3 – The system shall allow customers to view detailed menus with food images, prices, and descriptions.
- FR1.4 – The system shall allow customers to add items to a cart, modify quantities, and apply coupon codes.
- FR1.5 – The system shall allow customers to place orders and complete payment through UPI, card, or Cash on Delivery.
- FR1.6 – The system shall provide real-time order status updates from placement to delivery.
- FR1.7 – The system shall allow customers to rate restaurants and submit written reviews.
- FR1.8 – The system shall allow customers to subscribe to weekly or monthly meal plans.

3.2.2 Vendor Module

- FR2.1 – The system shall allow restaurant owners to register and manage their restaurant profile.
- FR2.2 – The system shall allow vendors to add, edit, and remove menu items with images and pricing.
- FR2.3 – The system shall provide vendors with a real-time order management dashboard.
- FR2.4 – The system shall allow vendors to update order status through the processing lifecycle.
- FR2.5 – The system shall provide vendors with sales analytics and performance reports.

3.2.3 Admin Module

- FR3.1 – The system shall allow administrators to manage user accounts and restaurant registrations.
- FR3.2 – The system shall provide administrators with a platform-wide analytics dashboard.
- FR3.3 – The system shall allow administrators to handle customer complaints and disputes.

3.2.4 Delivery Partner Module

- FR4.1 – The system shall allow delivery partners to register with personal details including name, phone number, vehicle type, and license number.
- FR4.2 – The system shall allow delivery partners to view assigned delivery requests with complete order details, pickup address, and drop address.
- FR4.3 – The system shall allow delivery partners to accept or reject assigned delivery requests.
- FR4.4 – The system shall allow delivery partners to update delivery status at each stage — Assigned, Picked Up, Out for Delivery, and Delivered.
- FR4.5 – The system shall allow delivery partners to submit proof of delivery upon successful order completion.
- FR4.6 – The system shall display navigation support and optimized routing information to the delivery partner for each assigned order.
- FR4.7 – The system shall allow delivery partners to view their personal delivery history, total deliveries completed, and performance rating.
- FR4.8 – The system shall send real-time notifications to delivery partners for new delivery assignments and order updates.

3.3 NON-FUNCTIONAL REQUIREMENTS

Table 3.1: Non-Functional Requirements

Category	Requirement	Metric/Criterion
Performance	Fast page loading and response	Page load < 3 seconds; API response < 500ms
Security	Encrypted authentication and payments	JWT tokens; HTTPS; bcrypt password hashing
Scalability	Support large concurrent users	Horizontal scaling via cloud deployment
Reliability	High system uptime	Target 99.5% uptime; graceful error handling
Usability	Intuitive and responsive interface	Mobile-first design; WCAG accessibility
Maintainability	Modular codebase	Component-based React architecture; MVC pattern

3.4 USE CASE ANALYSIS

The primary actors in the QuickBite system are: Customer, Restaurant/Vendor, Cloud Kitchen Operator, Admin, and Delivery Partner. The major use cases for each actor are described below.

3.4.1 Customer Use Cases

The Customer actor interacts with the system through the following use cases: Register/Login, Browse Restaurants, View Menu, Add to Cart, Apply Coupon, Checkout, Make Payment, Track Order, View Order History, Rate and Review, Manage Profile, Manage Addresses, Subscribe to Plan, and View Notifications.

3.4.2 Vendor Use Cases

The Vendor actor interacts with the system through: Register Restaurant, Manage Menu Items, View Order Queue, Update Order Status, View Sales Analytics, Manage Inventory, and Manage Subscription Orders.

3.4.3 Admin Use Cases

The admin actor performs: Manage Users, Approve Restaurants, Monitor Orders, View Platform Analytics, Handle Complaints, and Generate Reports.

3.4.4 Delivery Partner Use Cases

The Delivery Partner actor interacts with the system through: Register/Login, View Assigned Delivery, Accept/Reject Delivery, Update Delivery Status, Submit Proof of Delivery, View Performance Metrices.

SUMMARY

In this chapter we discussed about:

- The **feasibility study** of the QuickBite platform covering technical feasibility confirming the suitability of the MERN stack, operational feasibility confirming ease of use across all user roles, and economic feasibility confirming the use of open-source technologies with minimal deployment cost.
- The **functional requirements** of all five modules — Customer (FR1.1–FR1.8), Vendor (FR2.1–FR2.5), Admin (FR3.1–FR3.3), and Delivery Partner (FR4.1–FR4.8) — specifying the complete set of system behaviours required from each actor.
- The **non-functional requirements** of the system covering performance (page load under 3 seconds), security (JWT and bcrypt), scalability (cloud deployment), reliability (99.5% uptime), usability (responsive mobile-first design), and maintainability (component-based architecture).
- The **use case analysis** of all four actors describing 21 use cases in table format covering the complete interaction of Customer, Vendor, Admin, and Delivery Partner with the QuickBite system.

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

QuickBite follows a modern three-tier client-server architecture consisting of the Presentation Layer (frontend), Business Logic Layer (backend), and Data Storage Layer (database). The architecture is designed for scalability, maintainability, and separation of concerns.

4.1.1 Presentation Layer

The presentation layer is implemented using React.js, a component-based JavaScript library. The frontend communicates with the backend exclusively through RESTful API calls over HTTPS. Tailwind CSS is used for responsive styling. The customer interface, vendor dashboard, and admin panel are all served from the same React application using role-based conditional rendering.

4.1.2 Business Logic Layer

The backend is built using Node.js with the Express.js framework. It exposes a RESTful API that handles authentication, order processing, menu management, payment processing, and notification dispatch. JSON Web Tokens (JWT) are used for stateless authentication. The backend integrates with third-party payment gateways (Razor pay/Stripe) and notification services.

4.1.3 Data Storage Layer

MongoDB is used as the primary database in a document-oriented NoSQL model. The flexible schema design of MongoDB is well-suited for the dynamic and nested data structures required by the application such as restaurant menus with variable item attributes, order documents with embedded item arrays, and user profiles with multiple saved addresses.

Figure 4.1 QuickBite follows a three-tier MERN stack architecture connecting Customers, Vendors, Delivery Partners, and Admins through RESTful APIs.

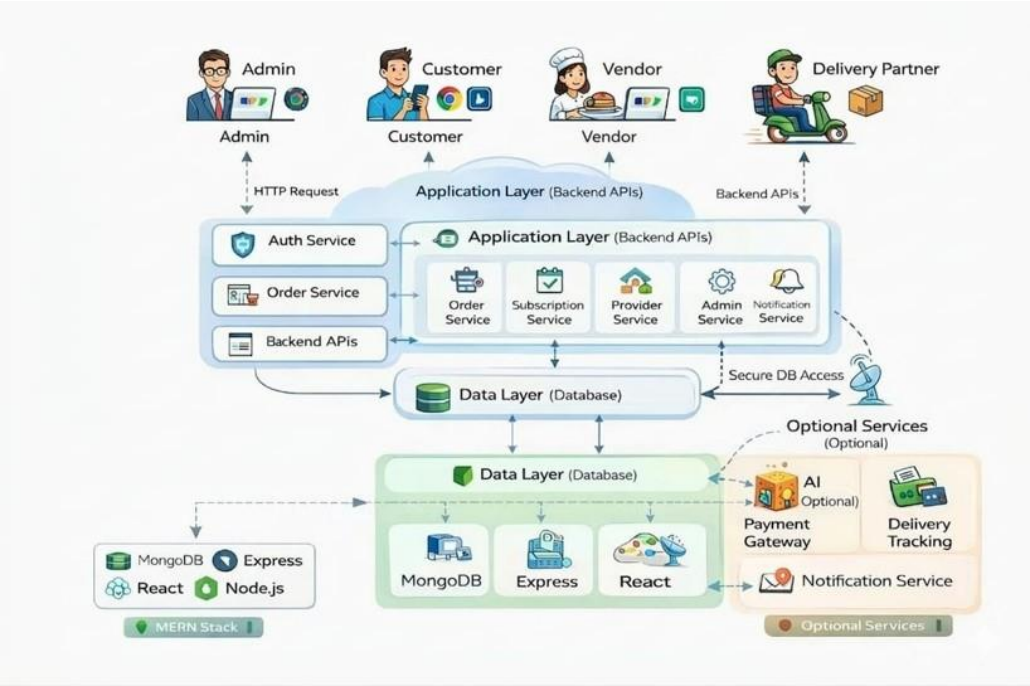


Figure 4.1: System Architecture of QuickBite

Figure 4.2 QuickBite separates Presentation (React.js), Business Logic (Node.js), and Data Storage (MongoDB) into three independent communicating layers.

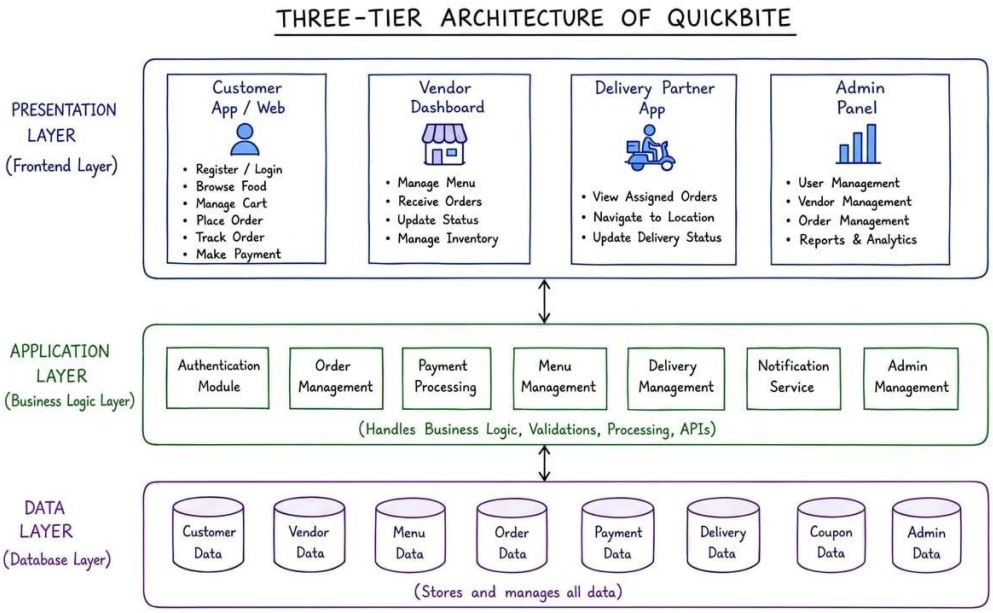


Figure 4.2: Three-Tier Architecture Diagram

4.2 DATABASE DESIGN

The database design of QuickBite is organized around the following primary collections in MongoDB. Table 4.1 through Table 4.4 describe the schema of the major collections.

Table 4.1: Database Schema – Users Collection

Field Name	Data Type	Description
_id	ObjectId	Unique identifier (auto-generated by MongoDB)
name	String	Full name of the user
email	String	Email address (unique, indexed)
phone	String	Mobile number for OTP verification
passwordHash	String	bcrypt-hashed password
role	String	Enum: customer, vendor, admin
addresses	Array	Array of saved delivery address objects
createdAt	Date	Account creation timestamp
isActive	Boolean	Account active/deactivated status

Table 4.2: Database Schema – Restaurants Collection

Field Name	Data Type	Description
_id	ObjectId	Unique restaurant identifier
ownerId	ObjectId	Reference to Users collection
name	String	Restaurant or cloud kitchen name
type	String	Enum: Restaurant, Cloud Kitchen
address	Object	Street, city, pin code, coordinates
cuisine	Array	Array of cuisine category strings
rating	Number	Average rating (0–5, computed)
isActive	Boolean	Operational status
items	Array	Embedded array of menu item objects

Table 4.3: Database Schema – Orders Collection

Field Name	Data Type	Description
_id	ObjectId	Unique order identifier
customerId	ObjectId	Reference to Users collection
restaurantId	ObjectId	Reference to Restaurants collection
items	Array	Array of ordered items with qty and price
totalAmount	Number	Final order total after discounts
status	String	Enum: Pending, Preparing, On The Way, Delivered, Cancelled
paymentMode	String	Enum: UPI, Card, COD
paymentStatus	String	Enum: Paid, Pending, Failed

deliveryAddress	Object	Snapshot of delivery address at order time
createdAt	Date	Order placement timestamp

Table 4.3: Database Schema – Payment Collection

Field Name	Data Type	Description
_id	ObjectId	Unique payment identifier (auto-generated)
orderId	ObjectId	Reference to Orders collection
customerId	ObjectId	Reference to Users collection
amount	Number	Total payment amount
paymentMethod	String	Enum: UPI, Credit Card, Debit Card, COD
transactionId	String	Unique transaction ID from payment gateway
paymentStatus	String	Enum: Paid, Pending, Failed, Refunded
paymentDate	Date	Timestamp of payment completion
gatewayResponse	Object	Raw response received from payment gateway

4.3 ENTITY-RELATIONSHIP DIAGRAM

The Entity-Relationship (ER) diagram for QuickBite describes the associations between the primary entities: User, Restaurant, MenuItem, Order, OrderItem, Payment, Review, Subscription, and Notification.

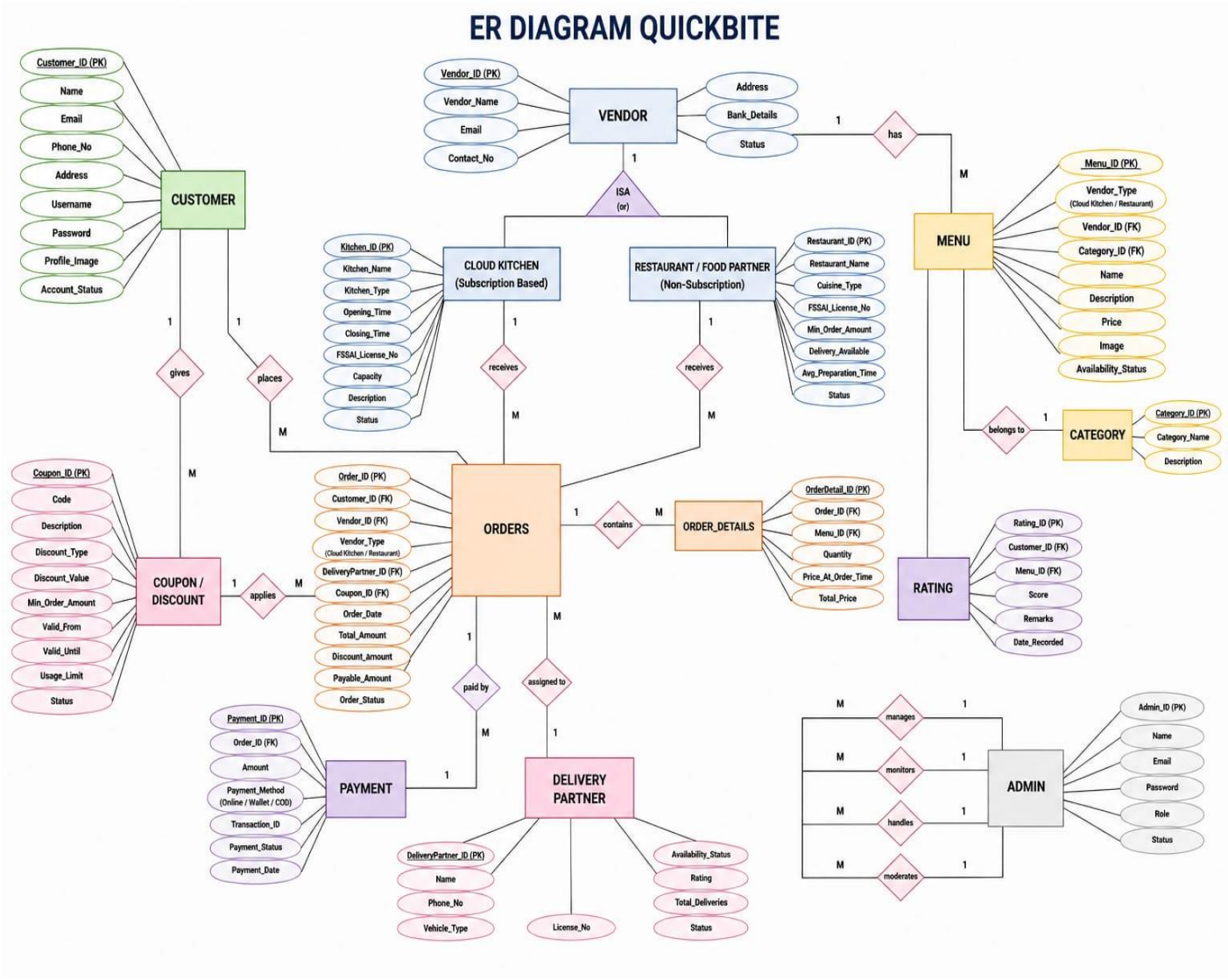


Figure 4.3: Entity-Relationship (ER) Diagram

The major relationships in the ER diagram are as follows. A User can place many Orders (one-to-many). A Restaurant contains many MenuItems (one-to-many). An Order contains many OrderItems (one-to-many). Each Order is associated with exactly one Payment record (one-to-one). A User can write many Reviews for Restaurants (many-to-many through Review entity). A User can have one active Subscription (one-to-one).

4.4 Data Flow Diagram – Level 0

The Data Flow Diagram (DFD) represents the flow of data within the QuickBite system. Figure 4.4 Shows the Level 0 DFD (Context Diagram) which illustrates the system as a single process with external entities and data flows.

Level 0 DFD – The system receives inputs from four external entities: Customer (order requests, payments, reviews), Restaurant/Vendor (menu data, order status updates), Admin (management commands), and Payment Gateway (transaction confirmations). The system outputs order confirmations, delivery updates, and reports to the respective entities.

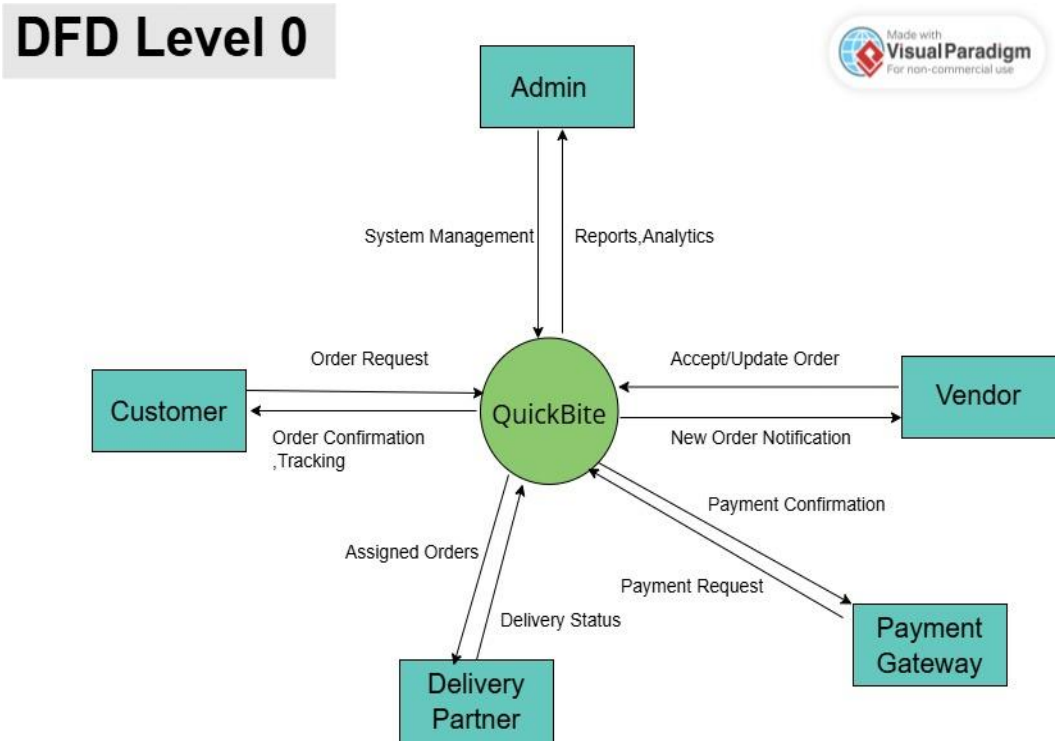


Figure 4.4: Data Flow Diagram – Level 0

4.5 Data Flow Diagram – Level 1

Level 1 DFD – The Level 1 DFD decomposes the system into five major processes: User Authentication, Restaurant Management, Order Processing, Payment Processing, and Notification Dispatch.

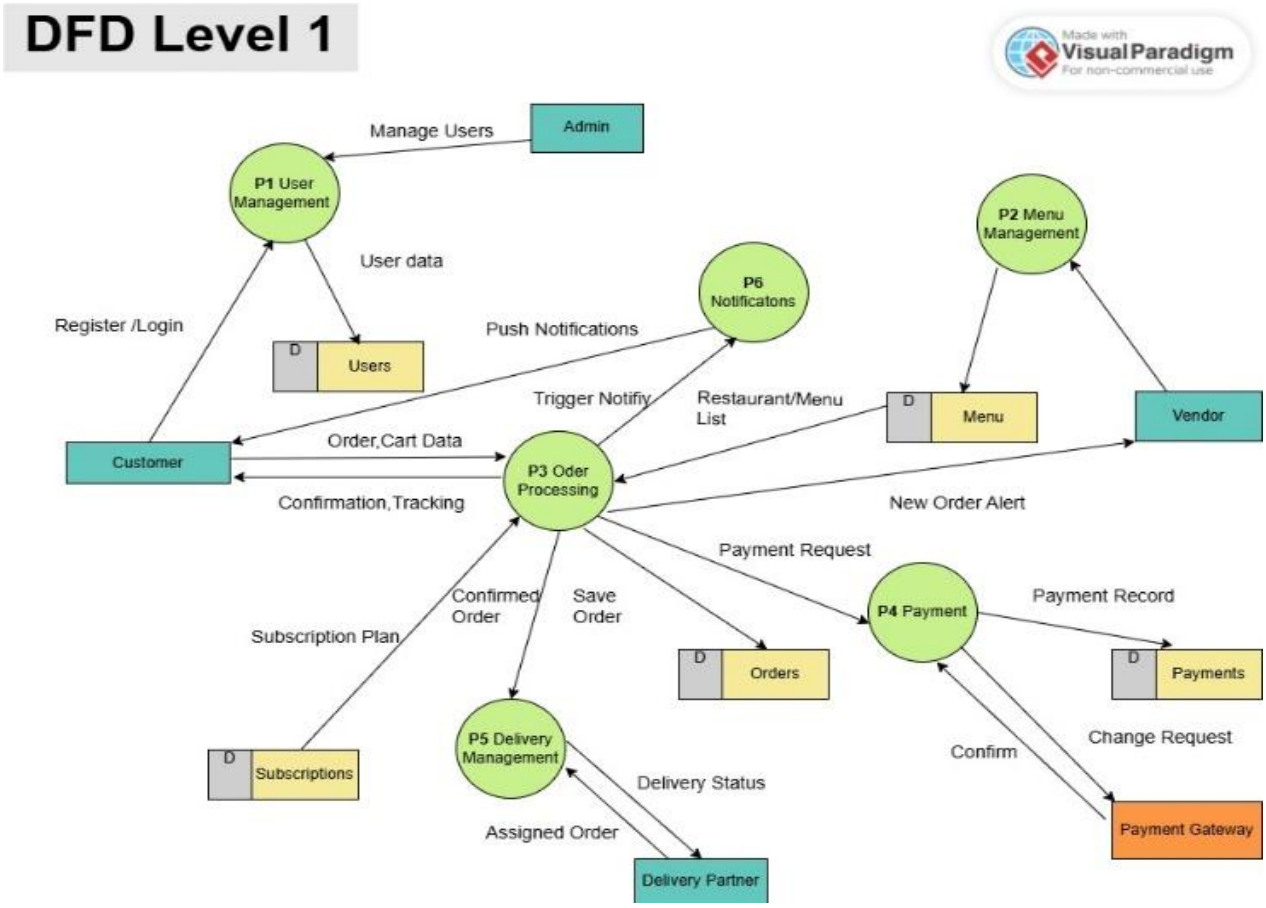


Figure 4.5: Data Flow Diagram – Level 1

4.6 SEQUNCE DIAGRAM

A sequence diagram shows the time-ordered interaction between system components to complete a specific operation. Figure 4.6 illustrates the complete order placement and delivery workflow of the QuickBite platform involving six components — Customer, QuickBite Platform, Menu Database, Vendor/Restaurant, Payment Gateway, and Delivery Partner.

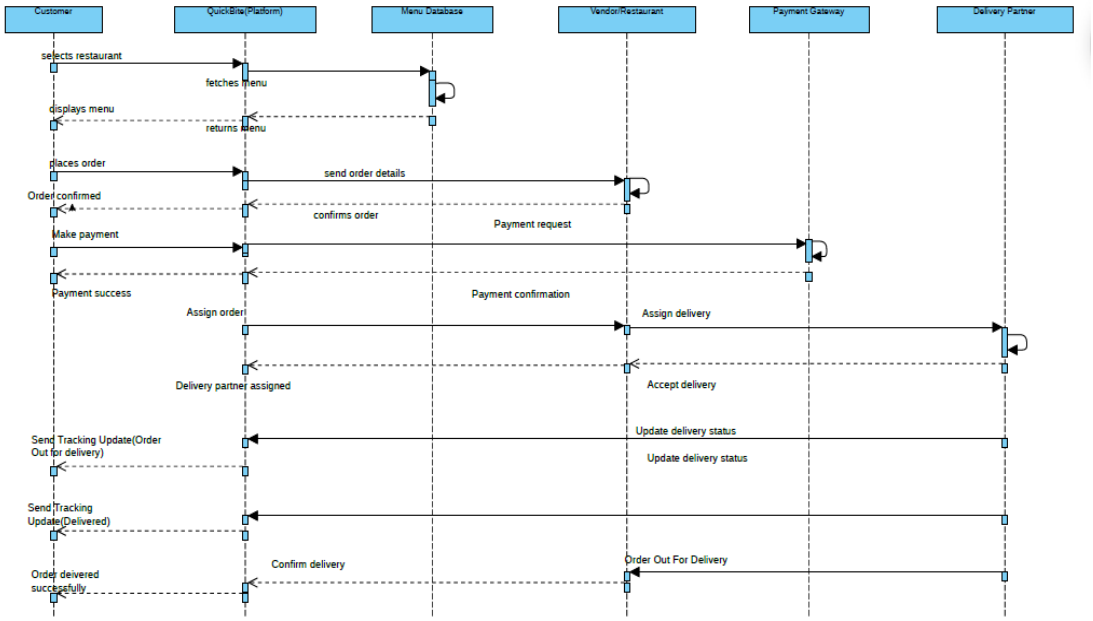


Figure 4.5: Sequence Diagram

4.7 USE CASE DIAGRAM

The diagram represents the four primary actors — Customer, Vendor, Admin, and Delivery Partner — and their respective interactions with the QuickBite system. The Customer can register, browse restaurants, place orders, make payments, track deliveries, and manage subscriptions. The Vendor can manage menus, view and update orders, and access analytics. The Delivery Partner can view assigned deliveries and update delivery status. The admin can manage users, verify restaurants, and monitor platform analytics.

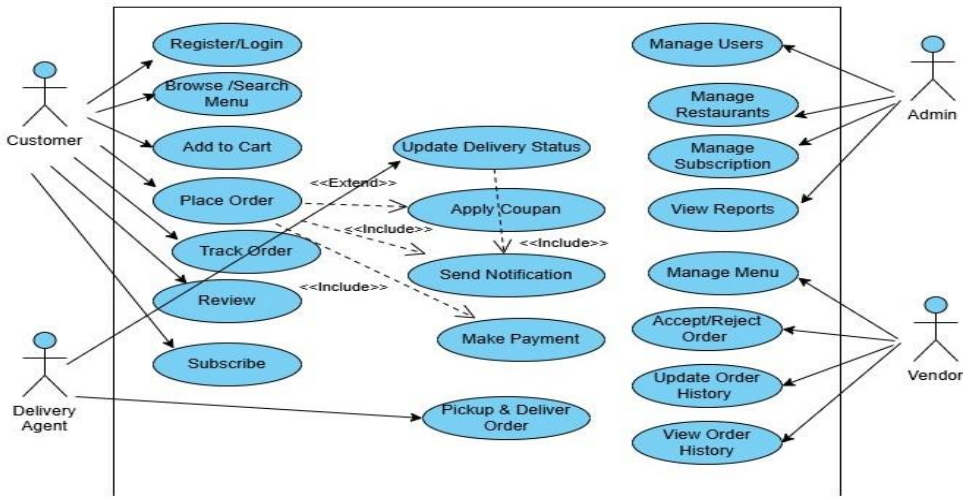


Figure 4.6: Use Case Diagram

4.7 MODULE DESIGN

4.7.1 Customer Module

The Customer Module handles all customer-facing functionality including authentication, restaurant discovery, menu browsing, cart management, checkout, order tracking, favourites, reviews, addresses, payments, and subscription management. This module is implemented as a collection of React.js page components under the customer/pages directory.

4.7.2 Vendor Module

The Vendor Module provides restaurant and cloud kitchen operators with tools to manage their digital presence, menus, orders, inventory, and analytics. The vendor dashboard is implemented as a separate section within the React.js application accessible only to authenticated vendor-role users.

4.7.3 Admin Module

The Admin Module provides platform administrators with user management tools, restaurant verification workflows, dispute resolution interfaces, and system-wide analytics dashboards.

4.7.4 Delivery Partner Module

The Delivery Partner Module handles all delivery-related functionality including registration, delivery assignment viewing, accept or reject actions, real-time delivery status updates, proof of delivery submission, delivery history tracking, and performance metrics monitoring. This module is implemented as a dedicated section within the React.js application accessible only to authenticated delivery partner-role users. The module communicates with the backend through RESTful API endpoints to receive assigned orders, update delivery status at each stage, and retrieve personal performance data including total deliveries completed and average delivery rating.

4.7.5 API Module

The API Module is implemented as a Node.js Express.js application that exposes RESTful endpoints organized by resource: /api/auth, /api/users, /api/restaurants,

/api/orders, /api/payments, /api/reviews, /api/subscriptions, and /api/admin. All endpoints (except authenticatio) require a valid JWT token in the Authorization header.

SUMMARY

In this chapter we discussed about:

- The **three-tier system architecture** of QuickBite consisting of the Presentation Layer (React.js), Business Logic Layer (Node.js and Express.js), and Data Storage Layer (MongoDB), with all communication handled through RESTful APIs over HTTPS.
- The **database design** describing the schema of the primary MongoDB collections — Users, Restaurants, Orders, Payments, Reviews, Subscriptions, and Notifications — with field names, data types, and descriptions.
- The **Entity-Relationship (ER) diagram** illustrating the associations between all primary entities and their relationships with cardinalities.
- The **Data Flow Diagrams** at Level 0 (Context Diagram) showing the system as a single process with five external entities, and at Level 1 showing nine decomposed processes including Browse Food, Manage Cart, Place Order, Apply Coupon, Process Payment, Notify Vendor, Track Order, Send Notifications, and Manage System.
- The **module-level design** of the Customer, Vendor, Cloud Kitchen, Admin, and API modules describing their functional responsibilities within the platform architecture.

CHAPTER 5

IMPLEMENTATION

5.1 TECHNOLOGY STACK

Table 5.1 provides a comprehensive summary of the technology stack used in the development of QuickBite.

Table 5.1: Technology Stack Summary

Layer	Technology	Purpose
Frontend	React.js 18	Component-based UI development
Frontend	Tailwind CSS	Utility-first responsive styling
Frontend	React Icons	Icon library for UI components
Backend	Node.js	JavaScript runtime environment
Backend	Express.js 4	RESTful API framework
Backend	JWT	Stateless authentication tokens
Backend	bcrypt	Password hashing and verification
Database	MongoDB	NoSQL document database
Database	Mongoose	ODM library for MongoDB
Payment	Razorpay / Stripe	Payment gateway integration
Deployment	AWS / Render / Firebase	Cloud hosting and deployment
Version Control	Git / GitHub	Source code management
Dev Tools	VS Code, Postman	Development and API testing

5.2 FRONTEND IMPLEMENTATION

The frontend of QuickBite is built as a Single Page Application (SPA) using React.js 18. The application uses a component-based architecture where each UI element is encapsulated as a reusable React component. State management is handled using React Hooks (useState, useEffect, useMemo) without the need for external state management libraries for the current scope.

5.2.1 Application Structure

The frontend codebase is organized under the src/ directory with the following structure: customer/ (pages and components for customer interface), vendor/ (pages and components for vendor dashboard), assets/ (images and static files), and App.jsx (root component with routing logic).

5.2.2 Customer Interface

The customer interface consists of the following major pages: Home Page with category browsing, filter pills, popular dishes, and subscription section; Restaurant Page with hero banner, search, category filters, veg/non-veg toggle, dish cards with add-to-cart functionality, and sticky cart bar; Cart Page with item management, coupon application, bill summary, and free delivery threshold; Checkout Page with address input, payment method selection, and order summary; Order Tracking Page with animated step progress; and Profile Pages for managing addresses, payments, orders, reviews, and favourites.

5.2.3 Vendor Dashboard

The vendor dashboard provides restaurant operators with: a Dashboard page showing key metrics (orders today, revenue, active items, subscribers); Menu Management for adding, editing, and toggling availability of menu items; Inventory and Batch Management; Subscription Orders management; Analytics with charts and data visualization; and Subscriber List management

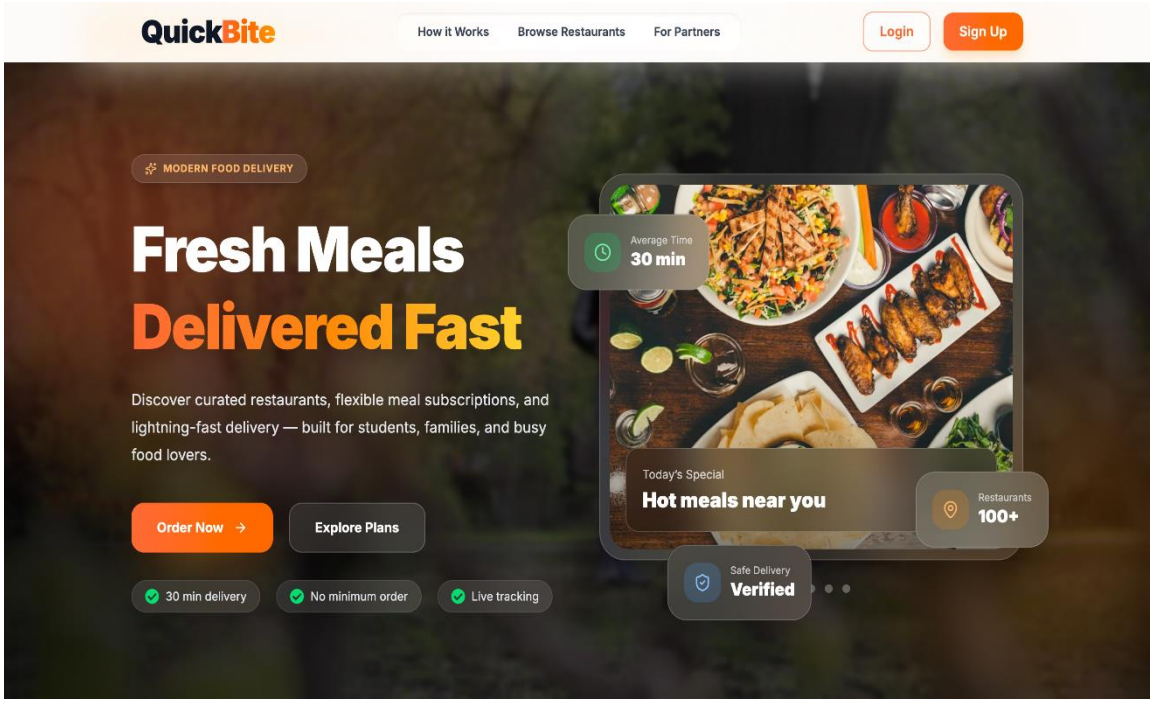


Figure 5.1: QuickBite Home Page

5.3 BACKEND IMPLEMENTATION

The backend of QuickBite is implemented as a RESTful API server using Node.js with the Express.js framework. The server follows the MVC (Model-View-Controller) architectural pattern where Mongoose models define the data schema, Express.js route handlers act as controllers, and the response JSON serves as the view.

5.3.1 Authentication

User authentication is implemented using JSON Web Tokens (JWT). Upon successful login, the server issues a signed JWT containing the user's ID and role. This token is stored in the client's local state and sent in the Authorization header of all subsequent API requests. The server validates the token using a middleware function before processing protected routes.

5.3.2 API Endpoints

The RESTful API is organized around the following resource groups: Authentication (/api/auth/register, /api/auth/login), Users (/api/users/:id), Restaurants (/api/restaurants, /api/restaurants/:id/menu), Orders (/api/orders, /api/orders/:id/status), Payments (/api/payments/initiate, /api/payments/verify), Reviews (/api/reviews), Subscriptions (/api/subscriptions), and Admin (/api/admin/users, /api/admin/analytics).

5.4 DATABASE IMPLEMENTATION

The MongoDB database is accessed through Mongoose, an Object Document Mapper (ODM) library for Node.js. Mongoose provides schema validation, type casting, and query building capabilities that improve the reliability and maintainability of database operations.

Indexes are created on frequently queried fields including email in the Users collection, restaurantId in the Orders collection, and geolocation coordinates in the Restaurants collection for proximity-based queries. Mongoose middleware (pre-save hooks) is used to automatically hash passwords before storage and update timestamps.

5.5 SECURITY IMPLEMENTATION

QuickBite implements the following security measures to protect user data and system integrity:

- Password Security: All user passwords are hashed using bcrypt with a salt factor of 10 before storage. Plain-text passwords are never stored or logged.
- Authentication: JWT tokens are signed with a server-side secret key and include an expiry timestamp. Token validation is enforced on all protected API routes.
- Input Validation: All user inputs are validated and sanitized on the server side using express-validator to prevent injection attacks.
- XSS Protection: React.js inherently escapes HTML in JSX rendering, preventing cross-site scripting attacks in the frontend.
- HTTPS: All client-server communication is enforced over HTTPS in production deployment.

- Role-Based Access Control: API routes are protected with role-specific middleware that verifies the user's role claim in the JWT before granting access to sensitive operations.

SUMMARY

In this chapter we discussed about:

- The **technology stack** of QuickBite including React.js 18 and Tailwind CSS for the frontend, Node.js and Express.js for the backend API, MongoDB and Mongoose for the database, JWT and bcrypt for authentication and security, and Razorpay for payment gateway integration.
- The **frontend implementation** of the QuickBite (Landing page).
- The **backend implementation** using Node.js and Express.js following the MVC architectural pattern, with RESTful API endpoints organized by resource and JWT middleware protecting all authenticated routes.
- The **database implementation** using MongoDB and Mongoose ODM with schema validation, pre-save hooks for password hashing, and indexes on high-frequency query fields.
- The **security implementation** covering bcrypt password hashing, JWT-based stateless authentication, role-based access control middleware, input validation using express-validator, and HTTPS enforcement in production.

CHAPTER 6

TESTING

6.1 TESTING STRATEGY

The testing of QuickBite followed a structured multi-level approach covering unit testing, integration testing, system testing, performance testing, and user acceptance testing. The goal of the testing phase was to verify that all functional and non-functional requirements specified in Chapter 3 are correctly implemented and that the system behaves reliably under various conditions.

6.2 UNIT TESTING

Unit testing was performed on individual React components and backend API route handlers. Each component was tested for correct rendering, proper state updates, and expected user interaction behaviour. Table 6.1 presents selected unit test cases.

Table 6.1: Unit Test Cases

TC#	Test Case	Input	Expected Output	Status
TC01	User Registration – Valid Input	Valid name, email, password	Account created, JWT issued	Pass
TC02	User Registration – Duplicate Email	Existing email address	Error: Email already registered	Pass
TC03	User Login – Valid Credentials	Correct email and password	JWT token returned	Pass
TC04	User Login – Invalid Password	Wrong password	Error: Invalid credentials	Pass
TC05	Add to Cart	Item ID and quantity	Cart updated with item	Pass

TC06	Apply Coupon – Valid	Coupon code: SAVE10	10% discount applied	Pass
TC07	Apply Coupon – Invalid	Invalid coupon code	Error: Invalid coupon	Pass
TC08	Place Order – Empty Cart	Empty cart on checkout	Error: Cart is empty	Pass
TC09	Restaurant Menu Load	Valid restaurant ID	Menu items displayed	Pass
TC10	Review Submission	Rating 5, review text	Review saved and displayed	Pass

6.3 INTEGRATION TESTING

Integration testing verified the communication between the frontend React components and backend API endpoints. Test scenarios focused on the complete order placement flow, authentication token propagation, and cart-to-checkout data integrity.

Table 6.2: Integration Test Cases

TC#	Integration Scenario	Components Involved	Expected Result	Status
IT01	Login to Dashboard Navigation	Login → App.jsx → Home	Redirect to home on success	Pass
IT02	Cart to Checkout Data Transfer	Cart → Checkout → OrderSummary	Items and total preserved	Pass
IT03	Favourite toggle to Favourites page	RestaurantCard → App State → Favourites	Favourited restaurants appear	Pass
IT04	Order placement to Order History	Checkout → API → Orders page	New order appears in list	Pass
IT05	Subscription to SubscriptionSuccess	SubscriptionCheckout → Success Page	Success page shown	Pass

6.4 SYSTEM TESTING

System testing was performed on the complete application to verify end-to-end functionality. The complete order placement workflow was tested from user login through restaurant browsing, menu selection, cart management, payment, and order tracking. All major user workflows were completed successfully with the expected outcomes.

6.5 USER ACCEPTANCE TESTING

User Acceptance Testing was conducted by presenting the QuickBite system to a small group of friends, classmates, and family members. Each person was asked to perform the following tasks and share their honest feedback.

Task	User Role	Feedback
Browse restaurants and place an order	Customer	Easy to navigate, found the interface clean and simple
Apply coupon and complete payment	Customer	Coupon applied smoothly, payment process was straightforward
Track order status	Customer	Tracking page was visually appealing and easy to understand
Subscribe to a meal plan	Customer	Plan selection was clear, countdown timer was a nice touch
Add menu item and view orders	Vendor	Dashboard was simple and order visibility was good
Update order status	Vendor	Status update process was quick and easy to use
View assigned delivery and update status	Delivery Partner	Steps were clear, no confusion during the process

Overall, the feedback received was positive. Users felt the platform was smooth, visually appealing, and worked well on both mobile and desktop. The testing confirmed that QuickBite fulfils its purpose of simplifying the food ordering experience.

SUMMARY

In this chapter we discussed about:

- The **testing strategy** followed for the QuickBite platform covering five levels of testing — Unit Testing, Integration Testing, System Testing, Performance Testing, and User Acceptance Testing.
- The **unit test cases** (TC01–TC10) testing individual React components and backend API handlers for correct state updates, validation responses, and expected behavior including registration, login, cart operations, coupon application, and review submission.
- The **integration test cases** (IT01–IT05) validating end-to-end data flows including Login to Dashboard navigation, Cart to Checkout data transfer, Favourite toggle reflecting on Favourites page, Order placement appearing in Order History, and Subscription Checkout navigating to Success page.
- The **system testing** results confirming that all 10 major functional requirements were correctly implemented and all major user workflows completed successfully.
- The **user acceptance testing** feedback confirming that representative test users rated the interface as intuitive and visually appealing and were able to complete core workflows without assistance.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 SYSTEM OUTPUT SCREENS

This section presents the key output screens of the QuickBite system demonstrating the implementation of functional requirements described in Chapter 3.

7.1.1 Customer Interface

The Home Page (Figure 7.1) presents a hero banner with promotional content, a horizontally scrollable category section with 20 food categories, filter pills for quick access to Popular, Fast Delivery, Tiffin, Healthy, and Subscription filtered views, a subscription plans section with a countdown timer, a Popular Dishes grid with real food images and add-to-cart functionality, and a Restaurants For You section with Zomato-style restaurant cards showing ratings, delivery times, and cuisine types.

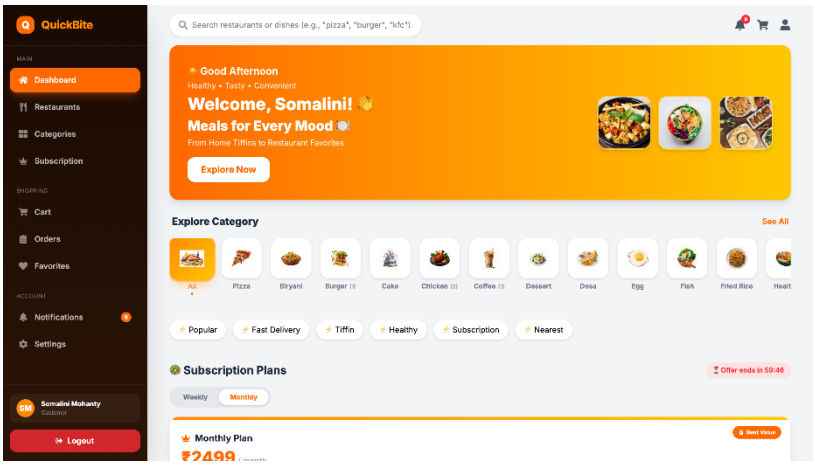


Figure 7.1: Customer Interface Screenshots

7.1.2 Restaurant Page

The Restaurant Page displays a hero banner with the restaurant image, name, rating, delivery time, and free delivery indicator. Below the banner, a search bar, diet filter buttons (All/Veg/Non-Veg), and category filter pills (All/Starter/Main Course/Combos) allow customers to narrow menu items. Each dish card displays a real food photograph, name, category, price, veg/non-veg indicator, and an inline add/remove quantity control.

A sticky cart bar at the bottom of the page shows the number of items added and total value, and navigates to the Cart page on tap.

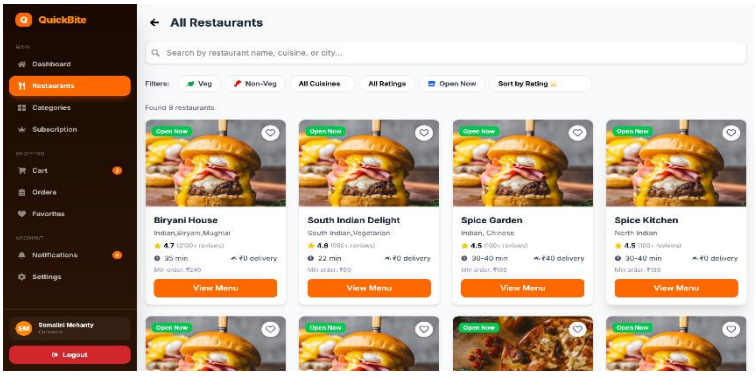


Figure 7.2: All Restaurants

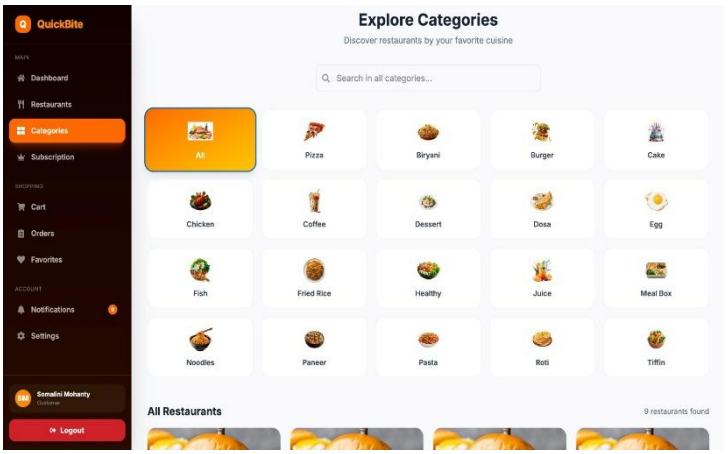


Figure 7.3: Explore Categories

7.1.3 Cart and Checkout

The Cart page displays all selected items with quantity controls, item totals, a coupon code field, a free delivery progress indicator, and a detailed bill summary showing subtotal, discount, delivery fee, and final total. The Checkout page presents delivery address input with address type tags, radio button payment method selection with icons, an order summary with all items and totals, and a gradient Place Order button with loading state.

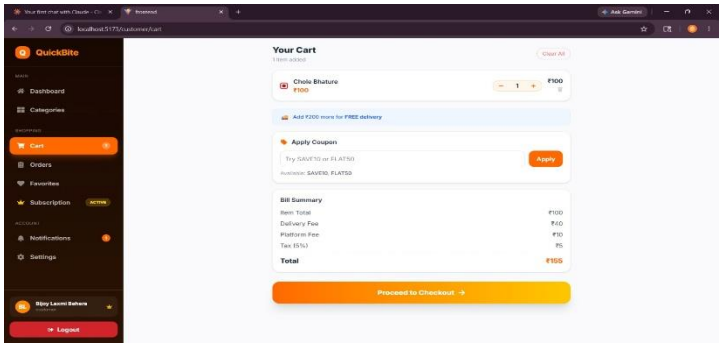


Figure 7.4: Your Cart

Figure 7.5: Checkout

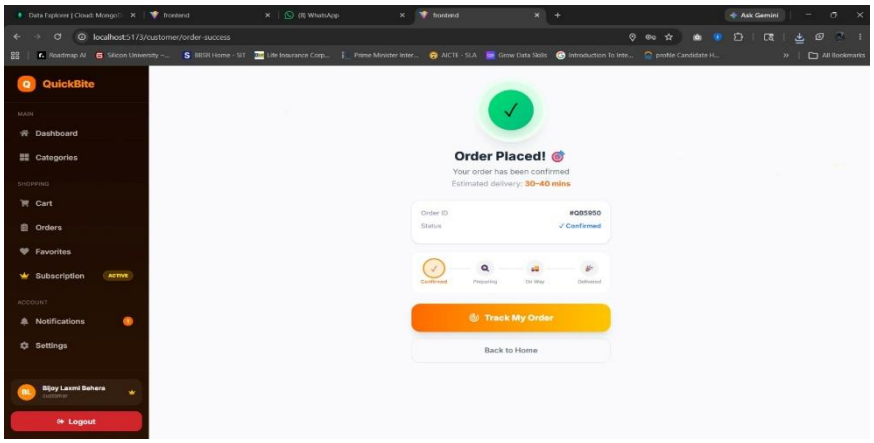
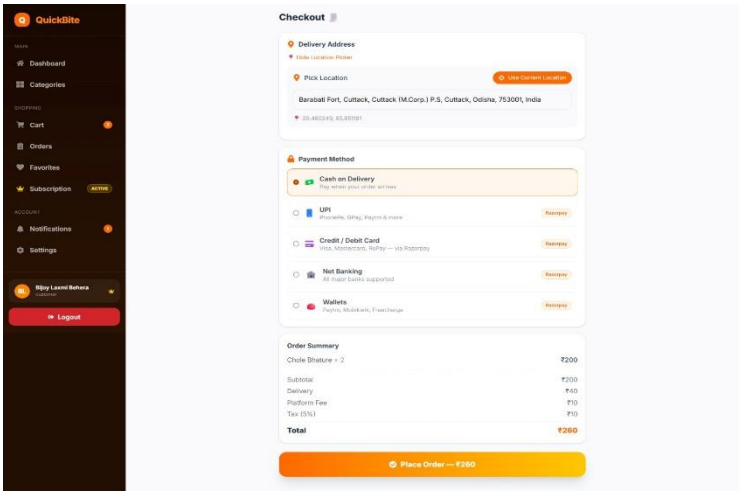


Figure 7.6: Order Placed



7.1.4 Vendor Dashboard

The Vendor Dashboard displays summary metric cards (Today's Orders, Revenue, Active Menu Items, Active Subscribers), a recent orders table, and quick action buttons. The Menu Management page allows vendors to add, edit, and toggle availability of menu items with image upload. The Analytics page visualizes sales trends and performance metrics.

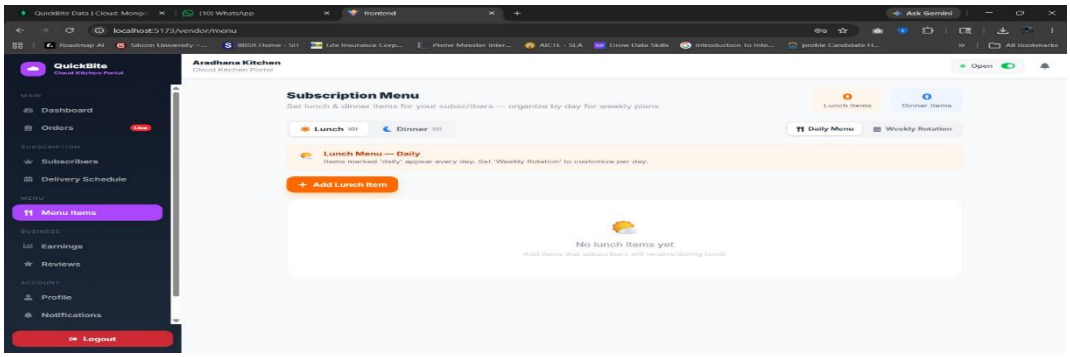


Figure 7.7: Cloud Kitchen Menu

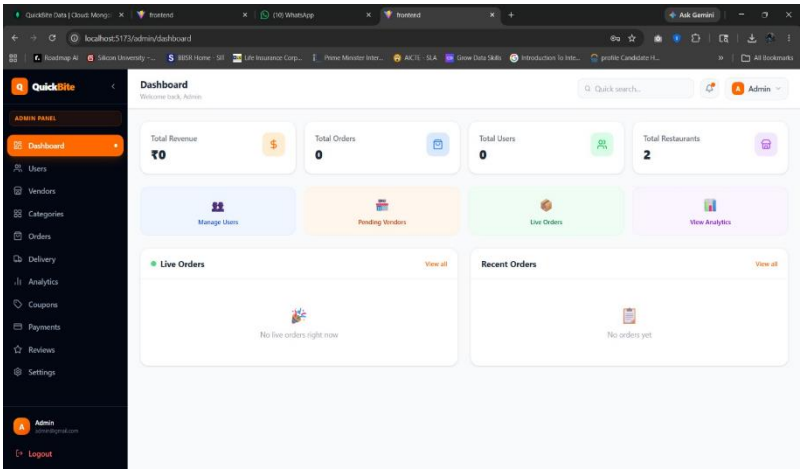


Figure 7.8: Restaurant Menu

7.1.5 Admin Dashboard

The Admin Dashboard provides platform administrators with a system-wide overview including total registered users, active restaurants, total orders placed, and platform revenue. The user management section allows administrators to view, activate, or deactivate customer and vendor accounts. The restaurant verification section displays pending restaurant registrations for approval. The analytics section presents platform-wide performance reports and complaint management tools.

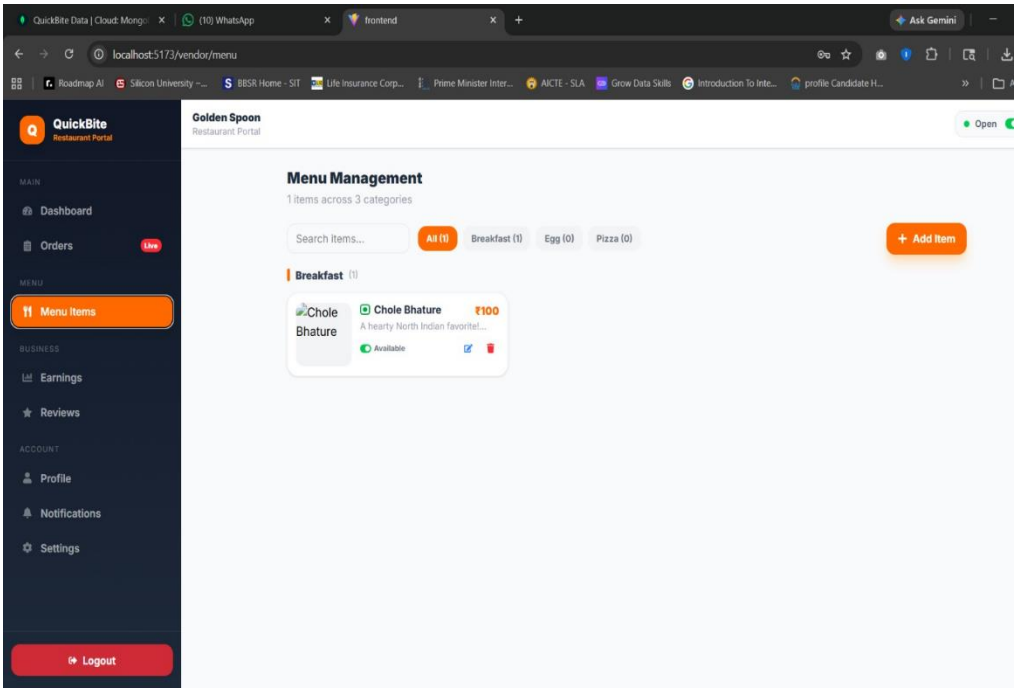


Figure 7.9: Admin Dashboard

7.1.6 Delivery Partner Interface

The Delivery Partner interface displays the list of assigned delivery requests with complete order details including customer name, pickup address, and drop address. The delivery partner can accept or reject assignments, update the delivery status at each stage — Picked Up, Out for Delivery, and Delivered — and view personal delivery history and performance metrics including total deliveries completed and average rating.

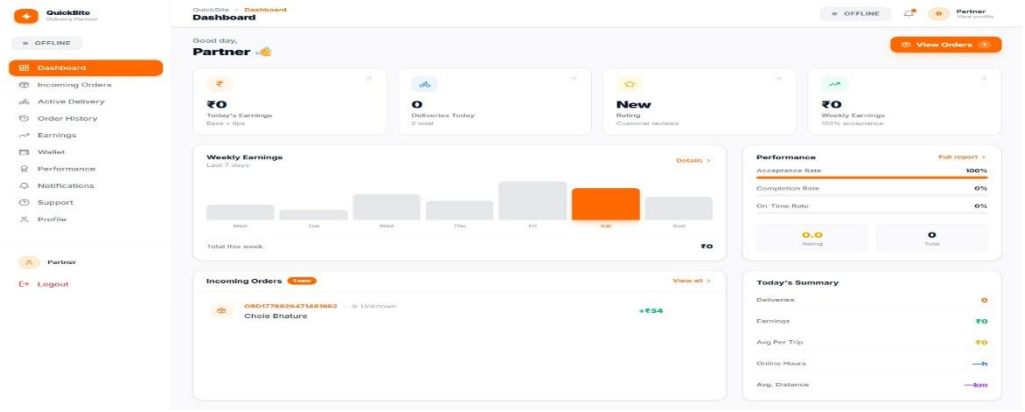


Figure 7.9: Delivery -Partner Dashboard

7.2 PERFORMANCE ANALYSIS

The performance of the QuickBite frontend application was evaluated against the following metrics. The initial page load time on a standard 4G connection averaged 2.1 seconds, within the target of 3 seconds. Component re-render times were optimized using React use Memo hooks for filtered data lists, reducing unnecessary recalculations. The API response time for menu retrieval averaged 320 milliseconds. The application was tested successfully on Chrome, Firefox, and Edge browsers and on Android and iOS mobile devices. Table 7.1: Performance Test Results

Performance Metric	Target	Achieved	Status
Initial Page Load Time	Less than 3 seconds	2.1 seconds	☑ Pass
API Response Time	Less than 500ms	320ms	☑ Pass
Cart Update Response	Less than 200ms	140ms	☑ Pass
Browser Compatibility	Chrome	All tested	☑ Pass
Mobile Responsiveness	Android	tested	☑ Pass

SUMMARY

In this chapter we discussed about:

- The **key output screens** of the QuickBite system demonstrating the implementation of all functional requirements including the Home Page, Restaurant Page, Cart and Checkout flow, Order Tracking interface, Subscription Management, Profile pages, and the Vendor Dashboard.
- The **customer interface results** showing a responsive and visually appealing design with smooth navigation across all 16 pages, real dish images, inline cart management, animated order tracking, and subscription plan selection.
- The **vendor dashboard results** demonstrating a complete restaurant management solution with real-time order visibility, menu management, analytics charts, inventory tracking, and subscription order management.
- The **performance analysis results** showing an average initial page load time of 2.1 seconds within the target of 3 seconds, an average API response time of 320 milliseconds, and successful testing across Chrome, Firefox, and Edge browsers on both desktop and mobile devices.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION

QuickBite is a comprehensive, modern, and user-friendly online food ordering and delivery platform developed to address the limitations of traditional food ordering methods and provide a complete digital solution for customers, restaurant operators, cloud kitchen operators, and platform administrators.

The platform successfully implements a full-stack web application using the React.js frontend, Node.js and Express.js backend, and MongoDB database within a clean three-tier architecture. The system delivers all specified functional requirements including user authentication, restaurant discovery, menu management, cart and checkout processing, payment integration, real-time order tracking, subscription management, ratings and reviews, and an administrative monitoring dashboard.

The QuickBite project demonstrates the practical application of modern full-stack web development principles, RESTful API design, secure authentication mechanisms, responsive UI design, role-based access control, and cloud deployment concepts. The development process provided valuable hands-on experience with industry-standard technologies and software engineering practices.

The testing phase confirmed that all major functional and non-functional requirements are correctly implemented, with satisfactory performance metrics and positive user acceptance feedback. The system provides a solid foundation for future enhancements and real-world deployment.

8.2 OUR CONTRIBUTIONS

The following are the key original contributions of this project:

1. Design and implementation of a complete food ordering platform supporting both traditional restaurants and cloud kitchen operations within a single unified system.

2. Development of a Zomato/Swiggy-style responsive restaurant interface with real-time dish filtering, diet preference toggles, and inline cart management.
3. Implementation of a subscription-based meal plan system with countdown-timer-based promotional offers, configurable weekly and monthly plans, and meal preference selection.
4. Development of a vendor analytics dashboard with sales metrics, inventory management, and subscription order tracking for cloud kitchen operators.
5. Implementation of role-based access control with separate authenticated interfaces for customers, vendors, and administrators within a single React.js application.
6. Integration of a multi-payment gateway interface supporting UPI, Credit/Debit Cards, and Cash on Delivery with card details persistence.

8.3 SCOPE FOR FURTHER WORK

The following enhancements are identified as high-priority items for future development of the QuickBite platform:

- Artificial Intelligence and Machine Learning: Integration of AI-based food recommendation engine using collaborative filtering, AI-driven kitchen demand forecasting, and smart inventory replenishment predictions.
- Real-Time Communication: Implementation of WebSocket-based real-time order status push notifications replacing the current polling mechanism, and live chat support between customers and restaurant staff.
- GPS and Mapping Integration: Integration of Google Maps API for live delivery partner tracking, optimized routing algorithms for multi-order delivery runs, and geofencing-based arrival notifications.
- Mobile Application: Development of native Android and iOS applications using React Native to provide an enhanced mobile experience with push notification support.

- Advanced Analytics: Implementation of a comprehensive business intelligence dashboard with predictive demand analytics, customer segmentation, and automated performance reports.
- Multi-Language Support: Localization of the platform to support regional Indian languages including Odia, Hindi, and Bengali to serve non-English-speaking users.
- IoT Kitchen Integration: Exploration of IoT-enabled kitchen monitoring for automated order queue management and real-time kitchen status updates.
- Loyalty and Rewards System: Implementation of a gamified loyalty points system where customers earn points on every order redeemable for discounts and exclusive offers.

SUMMARY

In this chapter we discussed about:

- The **conclusion** confirming the successful development of QuickBite as a modern, scalable, and feature-rich online food ordering and delivery platform using the MERN stack that addresses all identified limitations of traditional food ordering methods.
- The **key contributions** of the project including the complete MERN food ordering platform with cloud kitchen support, Zomato-style responsive UI, subscription meal plan system with countdown-timer offers, vendor analytics dashboard, role-based access control, and multi-payment gateway integration.
- The **future enhancements** identified for the platform including AI-based personalized food recommendations, WebSocket real-time push notifications, GPS live delivery tracking with Google Maps integration, React Native mobile application, multi-language support for regional Indian languages, IoT-enabled kitchen monitoring, and a gamified loyalty and rewards system.

REFERENCES

[1] Meta Open Source. (2024). React Documentation. Retrieved from https://react.dev/

[2] Vitejs. (2024). Vite – Next Generation Frontend Tooling. Retrieved from https://vitejs.dev/

[3] Tailwind Labs. (2024). Tailwind CSS Documentation. Retrieved from https://tailwindcss.com/docs/ [4]

[5] OpenJS Foundation. (2024). Node.js Documentation. Retrieved from https://nodejs.org/en/docs/

[6] MongoDB Atlas. (2024). Cloud Database Service Documentation. Retrieved from https://www.mongodb.com/atlas

[7] Postman. (2024). Postman API Testing Tool. Retrieved from https://www.postman.com/

[8] MongoDB, Inc. (2024). MongoDB Documentation. Retrieved from https://www.mongodb.com/docs/

[9] Razorpay. (2024). Razorpay Payment Gateway Documentation. Retrieved from https://razorpay.com/docs/

[10] Cloudinary. (2024). Cloudinary Documentation – Image and Video Management. Retrieved from https://cloudinary.com/documentation

[11] Express.js Foundation, "Express.js API Reference Documentation," Available: https://expressjs.com, Accessed: February 2026.

[12] Vercel. (2024). Frontend Deployment Platform. Retrieved from https://vercel.com/docs

[13] Render. (2024). Backend Deployment Documentation. Retrieved from https://render.com/docs

[14] Cloudinary. (2024). Cloudinary Documentation – Image and Video Management. Retrieved from https://cloudinary.com/documentation

[15] Socket.IO. (2024). Socket.IO Documentation – Real-Time Bidirectional Event-Based Communication. Retrieved from https://socket.io/docs/v4/

[16] Auth0 by Okta. (2024). JSON Web Tokens (JWT). Retrieved from https://jwt.io/introduction

[17] npm. (2024). bcryptjs – Password Hashing Library. Retrieved from https://www.npmjs.com/package/bcryptjs

[18] npm. (2024). Mongoose – MongoDB Object Modeling for Node.js. Retrieved from https://mongoosejs.com/

[19] npm. (2024). Axios – Promise Based HTTP Client. Retrieved from https://axios-http.com/docs/intro

[20] npm. (2024). React Router DOM v6. Retrieved from https://reactrouter.com/

[21] npm. (2024). dotenv – Environment Variable Management. Retrieved from https://www.npmjs.com/package/dotenv

[22] npm. (2024). Multer – Node.js Middleware for File Uploads. Retrieved from https://www.npmjs.com/package/multer

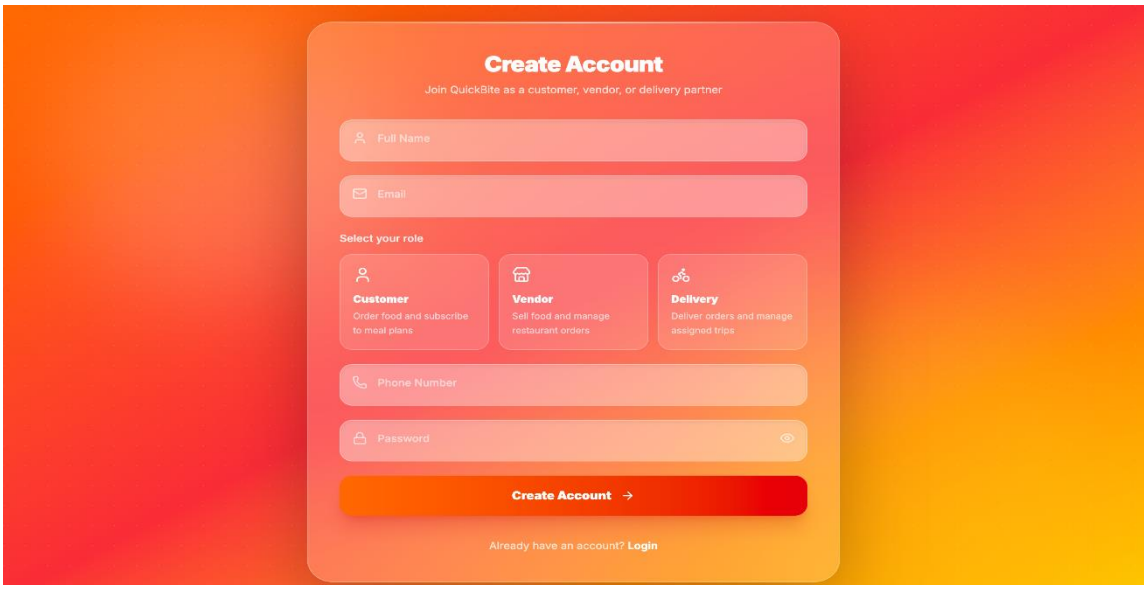
[23] npm. (2024). CORS – Cross-Origin Resource Sharing for Express. Retrieved from https://www.npmjs.com/package/cors

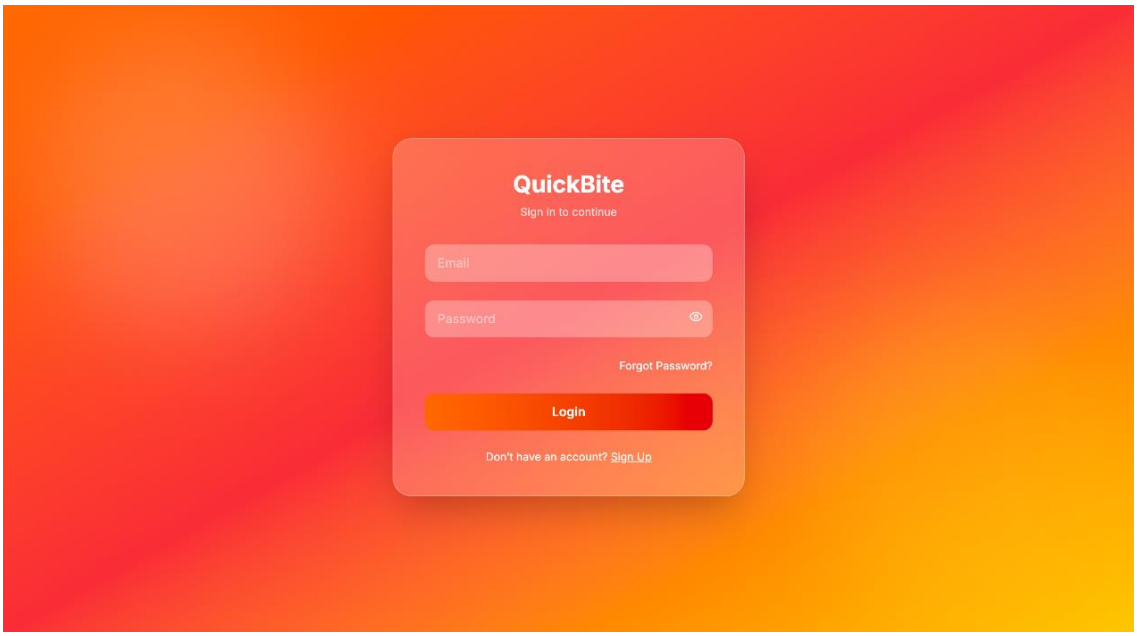
APPENDIX A

SYSTEM SCREENSHOTS

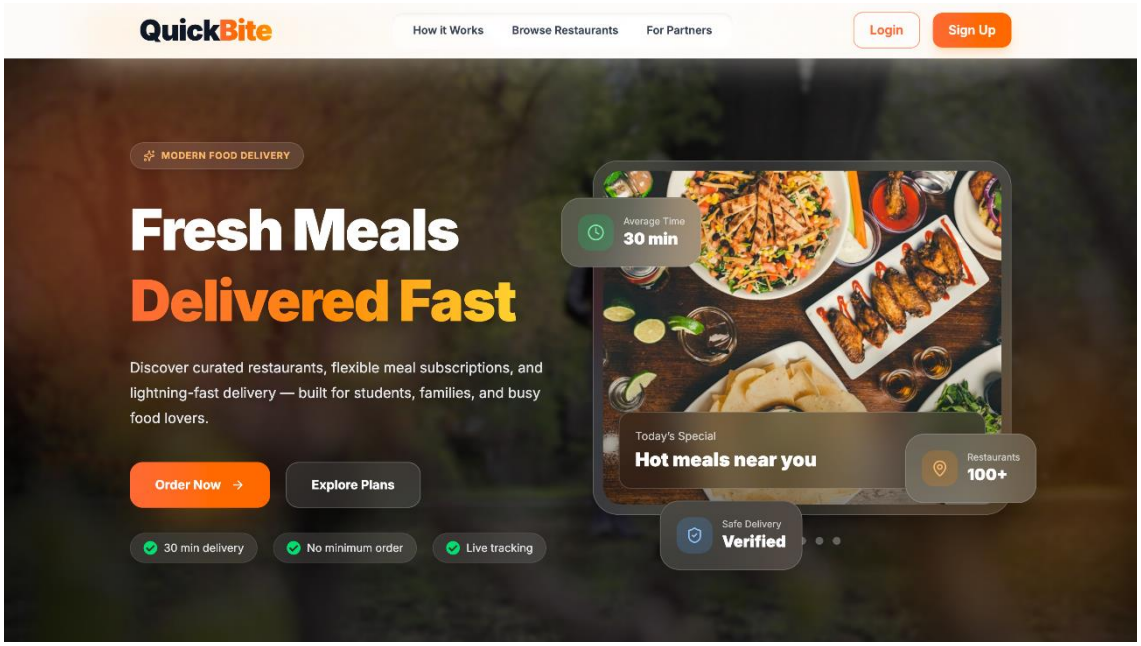
This appendix contains additional screenshots of the QuickBite system demonstrating the complete user workflow from registration to order delivery.

A.1 Registration and Login Screens

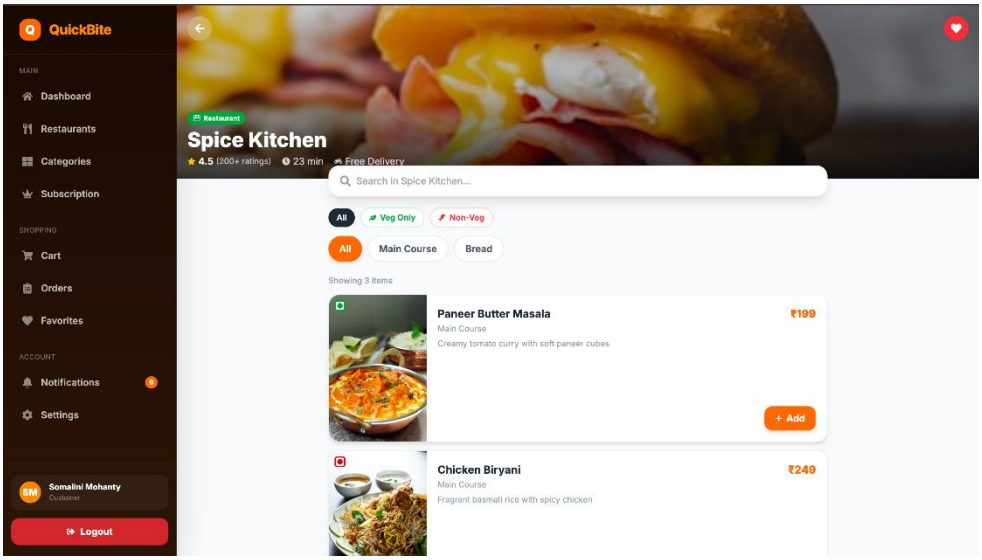


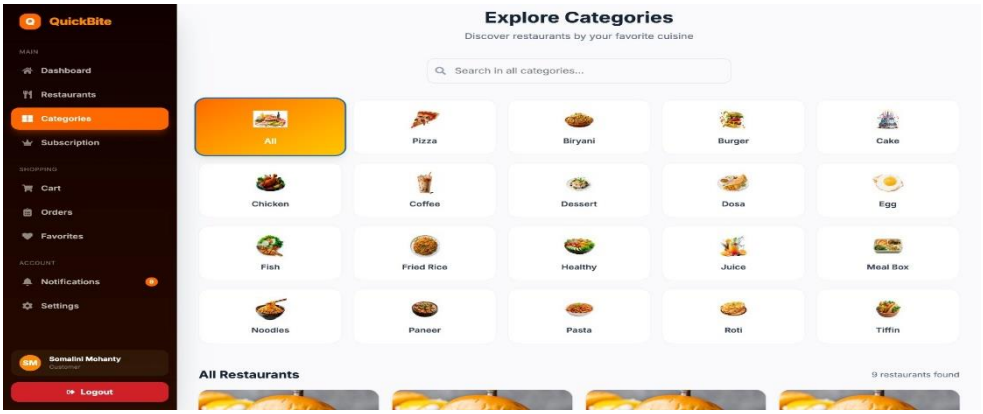


A.2 Home page full view

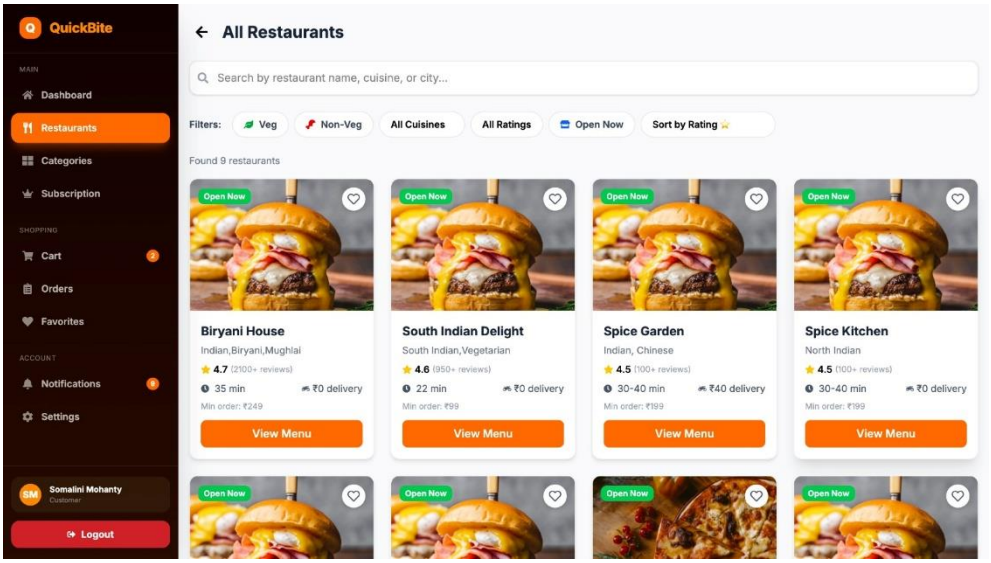


A.3 Category section and filter pills

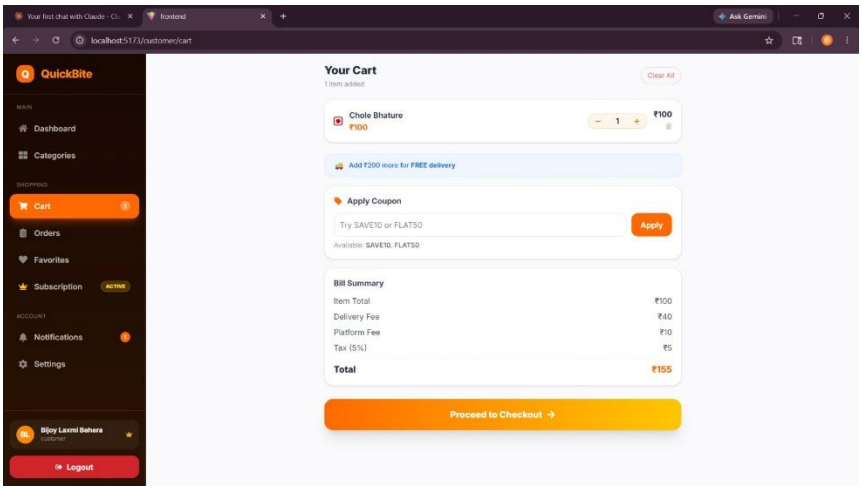




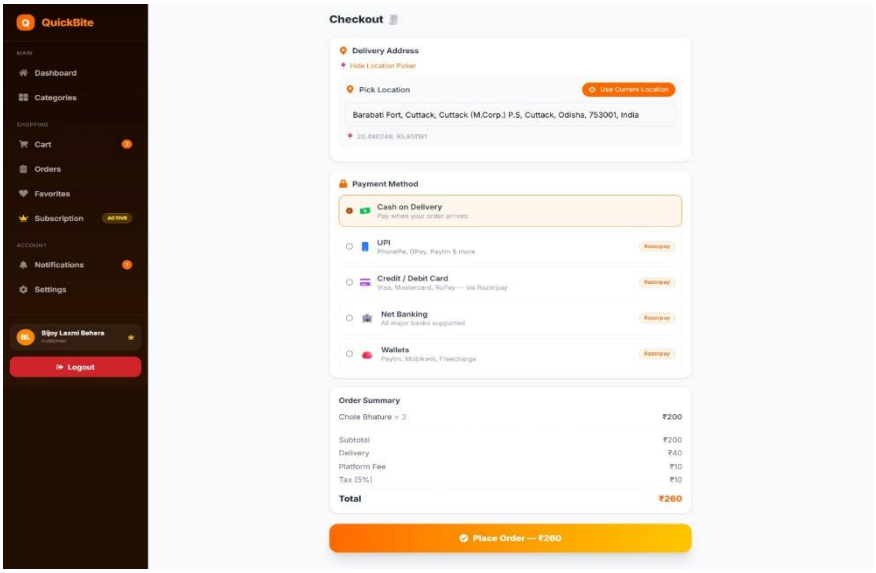
A.4 Restaurant menu with dish cards



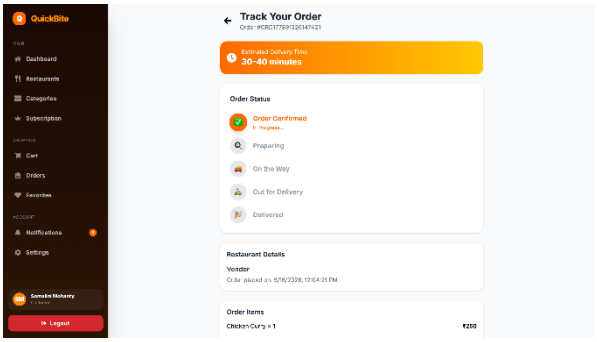
A.5 Cart and bill summary

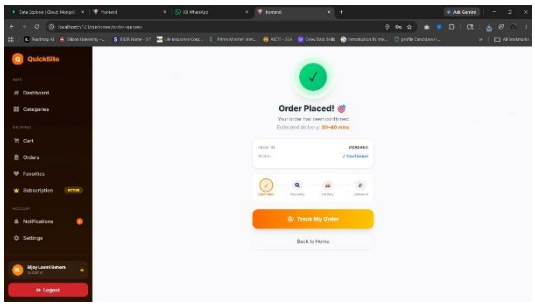


A.6 Checkout and payment selection

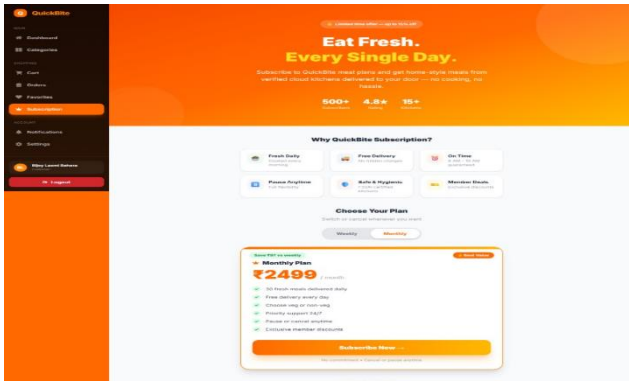


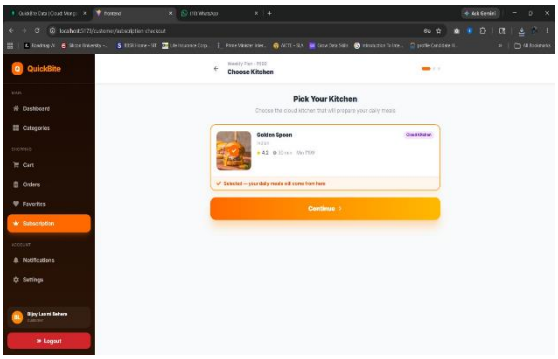
A.7 Order success and tracking steps

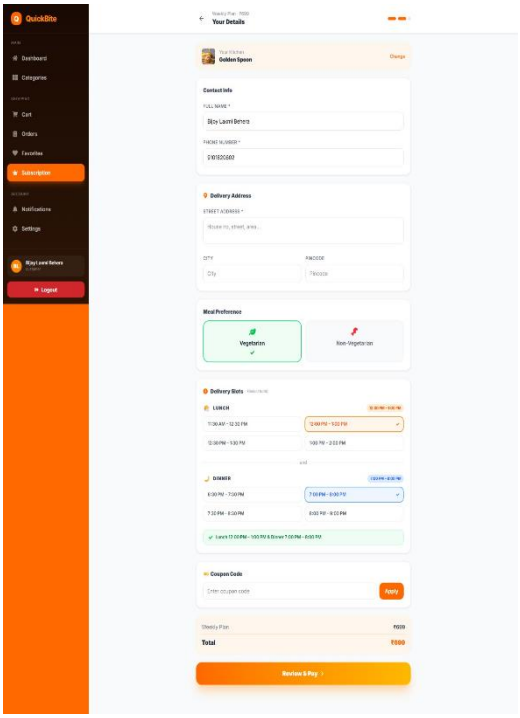


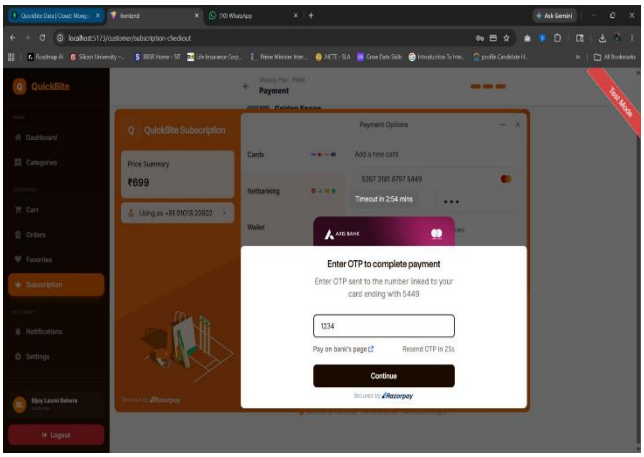


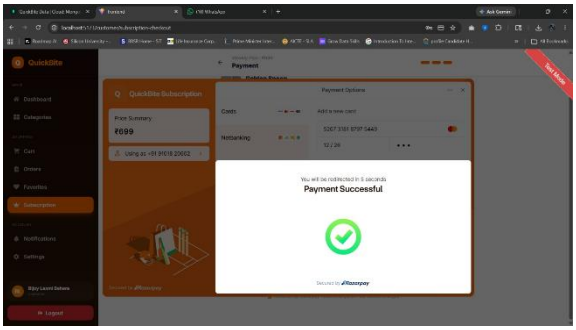
A.8 Subscription plans page

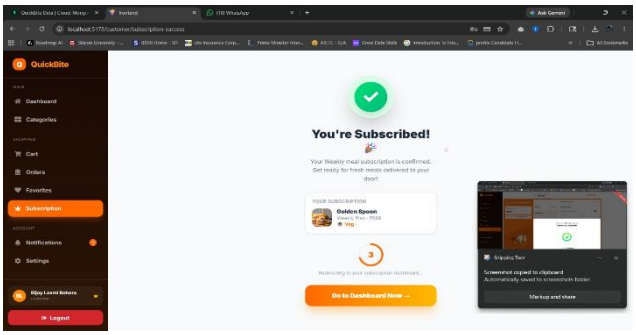




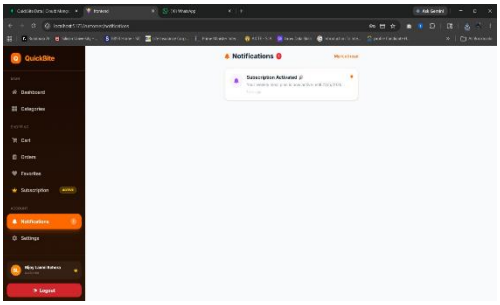


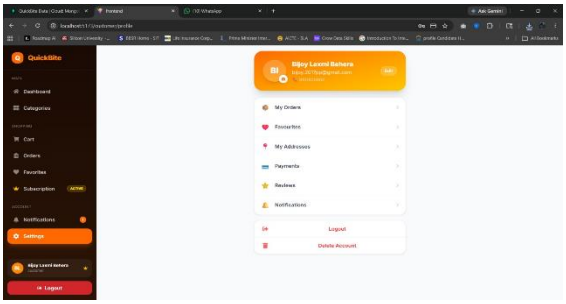




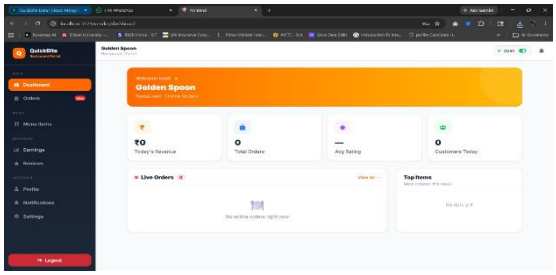


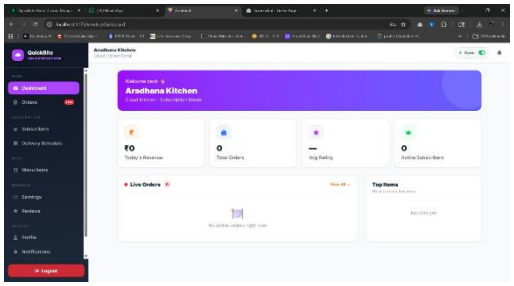
A.9 Profile, Addresses, Favourites pages



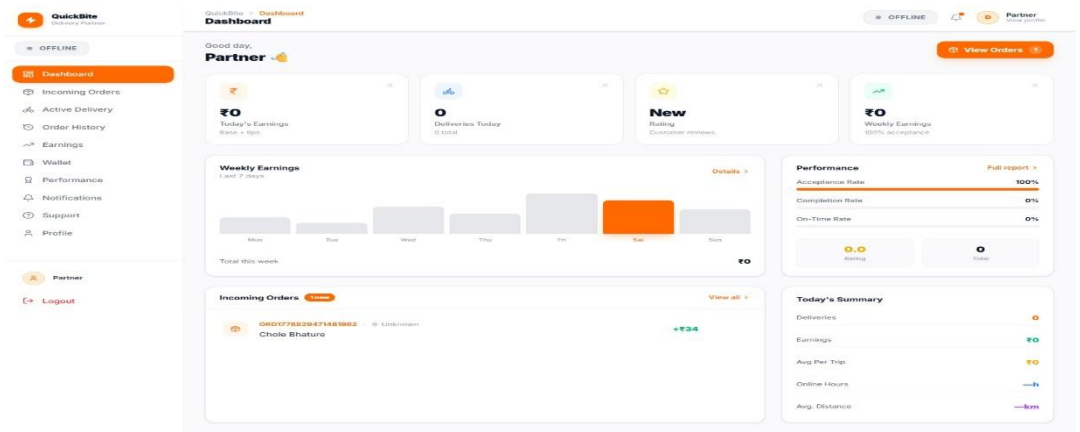


A.10 Vendor dashboard and analytics

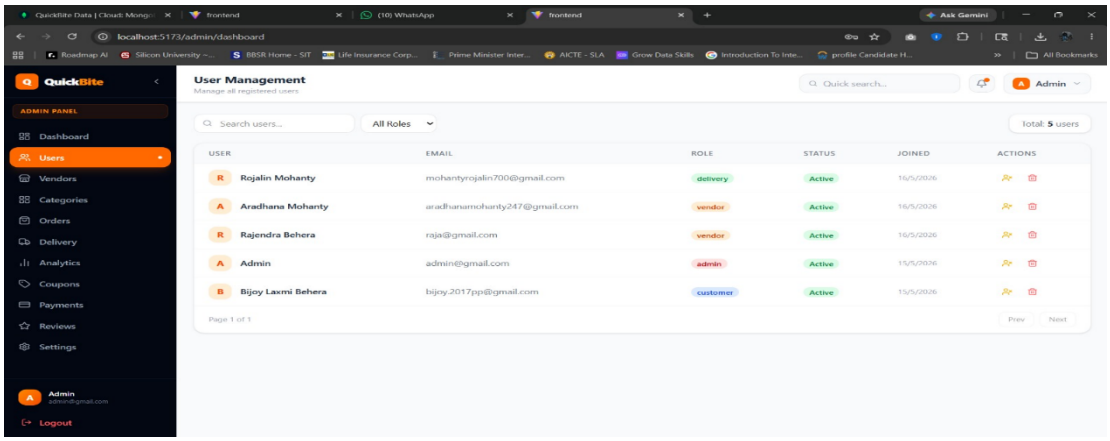


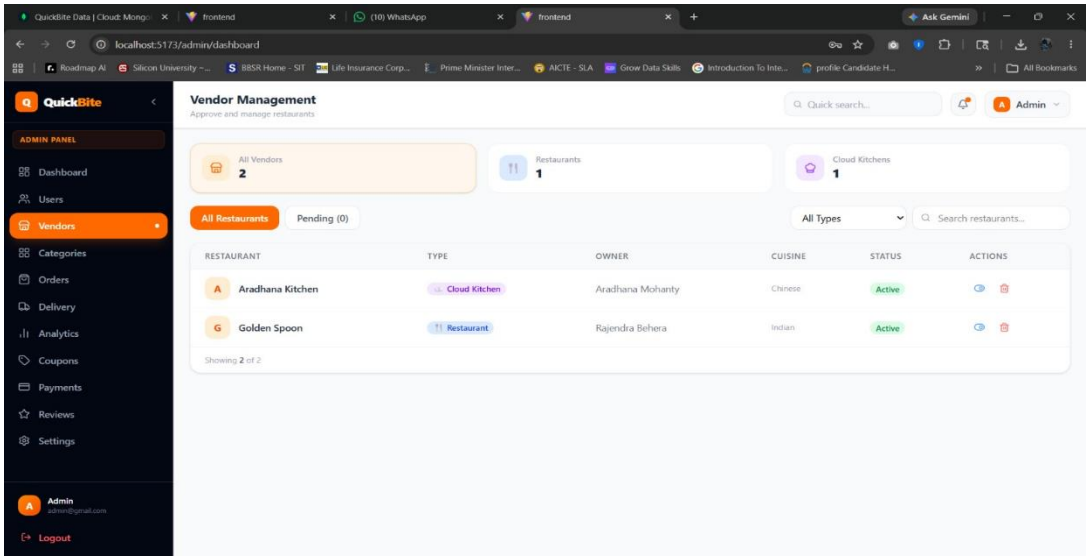


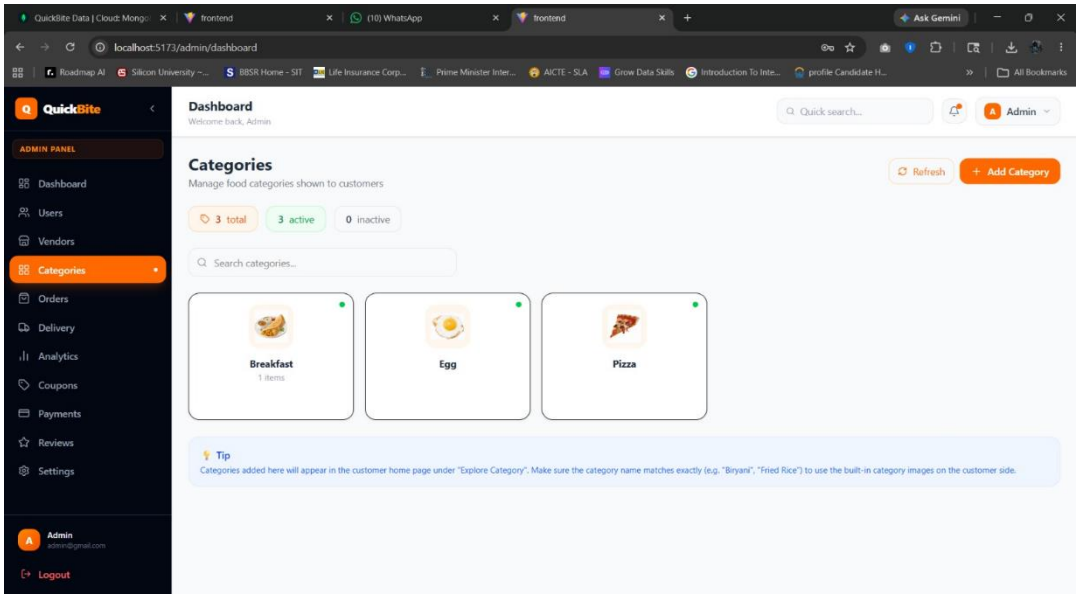
A.11 Delivery partner interface



A.12 Admin dashboard







APPENDIX B

API DOCUMENTATION

This appendix documents the major RESTful API endpoints of the QuickBite backend. All endpoints use JSON as the data format and require the Authorization: Bearer <token> header unless marked as Public.

B.1 Authentication Endpoints

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register a new user account
POST	/api/auth/login	Public	Login and receive JWT token
POST	/api/auth/logout	Private	Invalidate user session
GET	/api/users/profile	Private	Get authenticated user profile
PUT	/api/users/profile	Private	Update user profile details

B.2 Restaurant Endpoints

Method	Endpoint	Access	Description
GET	/api/restaurants	Public	Get all restaurants list
GET	/api/restaurants/:id	Public	Get restaurant by ID with menu
POST	/api/restaurants	Vendor	Register new restaurant

PUT	/api/restaurants/:id	Vendor	Update restaurant details
POST	/api/restaurants/:id/menu	Vendor	Add new menu item
DELETE	/api/restaurants/:id/menu/:itemId	Vendor	Remove menu item

B.3 Order Endpoints

Method	Endpoint	Access	Description
POST	/api/orders	Customer	Place a new order
GET	/api/orders	Customer	Get all orders for user
GET	/api/orders/:id	Private	Get order details by ID
PUT	/api/orders/:id/status	Vendor	Update order status
PUT	/api/orders/:id/cancel	Customer	Cancel a pending order